

async_scope – Creating scopes for non-sequential concurrency

Document #: P3149R8
Date: 2024-11-22
Project: Programming Language C++
Audience: SG1 Parallelism and Concurrency
LEWG Library Evolution
Reply-to: Ian Petersen
<ispeters@gmail.com>
Jessica Wong
<jesswong2011@gmail.com>
Contributors: Dietmar Kühl
<dkuhl@bloomberg.net>
Ján Ondrušek
<ondrusek@meta.com>
Kirk Shoop
<kirk.shoop@gmail.com>
Lee Howes
<lwh@fb.com>
Lucian Radu Teodorescu
<lucteo@lucteo.ro>
Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>

Contents

- 1 Changes
 - 1.1 R8
 - 1.2 R7
 - 1.3 R6
 - 1.3.1 The Problem
 - 1.3.2 Some Solutions
 - 1.3.3 LEWG Discussion in St Louis
 - 1.4 R5
 - 1.5 R4
 - 1.6 R3
 - 1.7 R2
 - 1.8 R1
 - 1.9 R0
- 2 Introduction
 - 2.1 Implementation experience
- 3 Motivation
 - 3.1 Motivating example
 - 3.2 `counting_scope` and `let_async_scope` are a step forward towards Structured Concurrency
- 4 Examples of use
 - 4.1 Spawning work from within a task
 - 4.2 Starting work nested within a framework
 - 4.3 Starting parallel work
 - 4.4 Listener loop in an HTTP server
 - 4.5 Pluggable functionality through composition

- 4.6 Recursively spawning work until completion
 - 4.6.1 `let_async_scope` with `spawn()`
 - 4.6.2 `let_async_scope` with `spawn_future()`
 - 4.6.3 `counting_scope`
- 5 Async Scope, usage guide
 - 5.1 Definitions
 - 5.2 `execution::async_scope_token`
 - 5.3 `execution::nest`
 - 5.4 `execution::spawn`
 - 5.5 `execution::spawn_future`
 - 5.6 `execution::simple_counting_scope`
 - 5.6.1 `simple_counting_scope::simple_counting_scope`
 - 5.6.2 `simple_counting_scope::~simple_counting_scope`
 - 5.6.3 `simple_counting_scope::get_token`
 - 5.6.4 `simple_counting_scope::close`
 - 5.6.5 `simple_counting_scope::join`
 - 5.6.6 `simple_counting_scope::token::wrap`
 - 5.6.7 `simple_counting_scope::token::try_associate`
 - 5.6.8 `simple_counting_scope::token::disassociate`
 - 5.7 `execution::counting_scope`
 - 5.7.1 `counting_scope::counting_scope`
 - 5.7.2 `counting_scope::~counting_scope`
 - 5.7.3 `counting_scope::get_token`
 - 5.7.4 `counting_scope::close`
 - 5.7.5 `counting_scope::request_stop`
 - 5.7.6 `counting_scope::join`
 - 5.7.7 `counting_scope::token::wrap`
 - 5.7.8 `counting_scope::token::try_associate`
 - 5.8 When to use `counting_scope` vs [P3296R2]’s `let_async_scope`
- 6 Design considerations
 - 6.1 Shape of the given sender
 - 6.1.1 Constraints on `set_value()`
 - 6.1.2 Handling errors in `spawn()`
 - 6.1.3 Handling stop signals in `spawn()`
 - 6.1.4 No shape restrictions for the senders passed to `spawn_future()` and `nest()`
 - 6.2 P2300’s `start_detached()`
 - 6.3 P2300’s `ensure_started()`
 - 6.4 Supporting the pipe operator
- 7 Naming
 - 7.1 `async_scope_token`
 - 7.2 `nest()`
 - 7.3 `spawn()`
 - 7.4 `spawn_future()`
 - 7.5 `simple_counting_scope`
 - 7.6 `counting_scope`
 - 7.6.1 `counting_scope::join()`

- 8 Specification
 - 8.1 Header <version> synopsis 17.3.2 [version.syn]
 - 8.2 Header <execution> synopsis 34.4 [execution.syn]
 - 8.3 Async scope concepts
 - 8.4 `execution::nest`
 - 8.5 `execution::spawn`
 - 8.6 `execution::spawn_future`
 - 8.7 `execution::simple_counting_scope` and `execution::counting_scope`
- 9 Acknowledgements
- 10 References

§ 1 Changes

§ 1.1 R8

- Replace `~async_scope_association()` with `async_scope_token.disassociate()` to address concerns raised during the LEWG meeting in Wrocław as captured in the polls below. The primary concern was the non-regularity of `async_scope_association`'s unusual copy constructor; requiring Standard Library implementers to remember to invoke `scopeToken.disassociate()` rather than relying on a non-regular RAI handle to do it automatically has more consensus.

POLL: We would like to change the spelling of the copy constructor of `async_scope_association`.

SF	F	N	A	SA
5	7	1	15	2

Attendance: [not recorded]

of Authors: 2

Authors' position: 2x A

Outcome: No consensus for change

SF: We don't have any copy ctor in the whole standard library that fails by silently not performing a copy.

SA: If we're not going to say this is a valid use of a copy ctor then we're saying this room doesn't believe in RAI.

POLL: Modify the spelling of the copy constructor of “`async_scope_association`” concept, without changing the copy constructor of “`nest`”.

SF	F	N	A	SA
1	6	2	1	2

Attendance: 30 IP + 6 (19 IP)

of Authors: 2

Author's Position: F + F

Outcome: Consensus in favor

SA: I was not permitted to ask questions about the poll

- Update the words of power regarding how various parts of the proposed types interact with the C++ memory model.

§ 1.2 R7

- Add wording to section 8.
- Remove the allocator from the environment in `spawn` and `spawn_future` when the allocator selection algorithm falls all the way back to using `std::allocator<>` because there's no other choice.
- Fix the last two typos in the example code.

— Small changes to how `spawn` and `spawn_future` set up the environment for the spawned operation based on feedback from Dietmar and Ruslan.

§ 1.3 R6

In revision 4 of this paper, Lewis Baker discovered a problem with using `nest()` as the basis operation for implementing `spawn()` (and `spawn_future()`) when the `counting_scope` that tracks the spawned work is being used to protect against out-of-lifetime accesses to the allocator provided to `spawn()`. Revision 5 of this paper raised Lewis’s concerns and presented several solutions. Revision 6 has selected the solution originally presented as “option 4”: define a new set of refcounting basis operations and define `nest()`, `spawn()`, and `spawn_future()` in terms of them.

§ 1.3.1 The Problem

What follows is a description, taken from revision 5, section 6.5.1, of the problem with using `nest()` as the basis operation for implementing `spawn()` (a similar problem exists for `spawn_future()` but `spawn()` is simpler to explain).

When a spawned operation completes, the order of operations was as follows:

1. The spawned operation completes by invoking `set_value()` or `set_stopped()` on a receiver, `rcvr`, provided by `spawn()` to the `nest-sender`.
2. `rcvr` destroys the `nest-sender`’s *operation-state* by invoking its destructor.
3. `rcvr` deallocates the storage previously allocated for the just-destroyed *operation-state* using a copy of the allocator that was chosen when `spawn()` was invoked. Assume this allocator was passed to `spawn()` in the optional environment argument.

Note that in step 2, above, the destruction of the `nest-sender`’s *operation-state* has the side effect of decrementing the associated `counting_scope`’s count of outstanding operations. If the scope has a `join-sender` waiting and this decrement brings the count to zero, the code waiting on the `join-sender` to complete may start to destroy the allocator while step 3 is busy using it.

§ 1.3.2 Some Solutions

Revision 5 presented the following possible solutions:

1. Do nothing; declare that `counting_scope` can’t be used to protect memory allocators.
2. Remove allocator support from `spawn()` and `spawn_future()` and require allocation with `::operator new`.
3. Make `spawn()` and `spawn_future()` basis operations of `async_scope_tokens` (alongside `nest()`) so that the decrement in step 2 can be deferred until after step 3 completes.
4. Define a new set of refcounting basis operations and define `nest()`, `spawn()`, and `spawn_future()` in terms of them.
5. Treat `nest-senders` as RAII handles to “scope references” and change how `spawn()` is defined to defer the decrement. (There are a few implementation possibilities here.)
6. Give `async_scope_tokens` a new basis operation that can wrap an allocator in a new allocator wrapper that increments the scope’s refcount in `allocate()` and decrements it in `deallocate()`.

§ 1.3.3 LEWG Discussion in St Louis

The authors opened the discussion by recommending option 6. By the end of the discussion, the authors’ expressed preferences were: “4 & 6 are better than 5; 5 is better than 3.” The biggest concern with option 4 was the time required to rework the paper in terms of the new basis operation.

The room took the following two straw polls:

1. In P3149R5 strike option 1 from 6.5.2 (option 1 would put the responsibility to coordinate the lifetime of the memory resource on the end user)

SF	F	N	A	SA
10	2	3	1	1

Attendance: 21 in-person + 10 remote

of Authors: 2

Authors’ position: 2x SF

Outcome: Consensus in favor

SA: I'm SA because I don't think async scope needs to protect memory allocations or resources, it's fine for this not to be a capability and I think adding this capability will add complexity, and that'll mean it doesn't make C++26.

2. In P3149R5 strike option 2 from 6.5.2 (option 2 would prevent spawn from supporting allocators)

SF	F	N	A	SA
8	4	2	2	0

Attendance: 21 in-person + 10 remote

of Authors: 2

Authors' position: 2x SF

Outcome: Consensus in favor

WA: As someone who was weakly against I'm not ready to rule out this possibility yet.

Ultimately, the authors chose option 4, leading to revision 6 of the paper changing from this:

```
template <class Token, class Sender>
concept async_scope_token =
    copyable<Token> &&
    is_nothrow_move_constructible_v<Token> &&
    is_nothrow_move_assignable_v<Token> &&
    is_nothrow_copy_constructible_v<Token> &&
    is_nothrow_copy_assignable_v<Token> &&
    sender<Sender> &&
    requires(Token token, Sender&& snd) {
        { token.nest(std::forward<Sender>(snd)) } -> sender;
    };
```

with `execution::nest()` forwarding to the `nest()` method on the provided token and `spawn()` and `spawn_future()` being expressed in terms of `nest()`, to this:

```
template <class Assoc>
concept async_scope_association =
    semiregular<Assoc> &&
    requires(const Assoc& assoc) {
        { static_cast<bool>(assoc) } noexcept;
    };

template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> async_scope_association;
    };
```

with `nest()`, `spawn()`, and `spawn_future()` all being expressed in terms of the `async_scope_token` concept.

§ 1.4 R5

- Clarify that the *nest-sender*'s operation state must destroy its child operation state before decrementing the scope's reference count.
- Add naming discussion.
- Discuss a memory allocator lifetime concern raised by Lewis Baker and several options for resolving it.

§ 1.5 R4

- Permit caller of `spawn_future()` to provide a stop token in the optional environment argument.
- Remove `[[nodiscard]]`.
- Make `simple_counting_scope::token::token()` and `counting_scope::token::token()` explicit and exposition-only.
- Remove redundant concept `async_scope`.
- Remove last vestiges of `let_async_scope`.
- Add some wording to a new [Specification](#) section

§ 1.6 R3

- Update slide code to be exception safe

- Split the async scope concept into a scope and token; update `counting_scope` to match
- Rename `counting_scope` to `simple_counting_scope` and give the name `counting_scope` to a scope with a stop source
- Add example for recursively spawned work using `let_async_scope` and `counting_scope`

§ 1.7 R2

- Update `counting_scope::nest()` to explain when the scope's count of outstanding senders is decremented and remove `counting_scope::joined()`, `counting_scope::join_started()`, and `counting_scope::use_count()` on advice of SG1 straw poll:

forward P3149R1 to LEWG for inclusion in C++26 after P2300 is included in C++26, with notes:

1. the point of refcount decrement to be moved after the child operation state is destroyed
2. a future paper should explore the design for cancellation of scopes
3. observers (`joined`, `join_started`, `use_count`) can be removed

SF	F	N	A	SA
10	14	2	0	1

Consensus

SA: we are moving something without wide implementation experience, the version with experience has cancellation of scopes

- Add a fourth state to `counting_scope` so that it can be used as a data-member safely

§ 1.8 R1

- Add implementation experience
- Incorporate pre-meeting feedback from Eric Niebler

§ 1.9 R0

- First revision

§ 2 Introduction

[P2300R7] lays the groundwork for writing structured concurrent programs in C++ but it leaves three important scenarios under- or unaddressed:

1. progressively structuring an existing, unstructured concurrent program;
2. starting a dynamic number of parallel tasks without “losing track” of them; and
3. opting in to eager execution of sender-shaped work when appropriate.

This paper describes the utilities needed to address the above scenarios within the following constraints:

- *No detached work by default*; as specified in [P2300R7], the `start_detached` and `ensure_started` algorithms invite users to start concurrent work with no built-in way to know when that work has finished.
 - Such so-called “detached work” is undesirable; without a way to know when detached work is done, it is difficult know when it is safe to destroy any resources referred to by the work. Ad hoc solutions to this shutdown problem add unnecessary complexity that can be avoided by ensuring all concurrent work is “attached”.
 - [P2300R7]’s introduction of structured concurrency to C++ will make async programming with C++ much easier but experienced C++ programmers typically believe that async C++ is “just hard” and that starting async work *means* starting detached work (even if they are not thinking about the distinction between attached and detached work) so adapting to a post-[P2300R7] world will require unlearning many deprecated patterns. It is thus useful as a teaching aid to remove the unnecessary temptation of falling back on old habits.
- *No dependencies besides [P2300R7]*; it will be important for the success of [P2300R7] that existing code bases can migrate from unstructured concurrency to structured concurrency in an incremental way so tools for progressively structuring code should not take on risk in the form of unnecessary dependencies.

The proposed solution comes in the following parts:

- `template <class Token> concept async_scope_token;`

- sender `auto` nest(sender `auto&&` snd, `async_scope_token` `auto` token);
- void spawn(sender `auto&&` snd, `async_scope_token` `auto` token, `auto&&` env);
- sender `auto` spawn_future(sender `auto&&` snd, `async_scope_token` `auto` token, `auto&&` env);
- Proposed in [P3296R2]: sender `auto` let_async_scope(callable `auto&&` senderFactory);
- `class` simple_counting_scope; and
- `class` counting_scope.

§ 2.1 Implementation experience

The general concept of an async scope to manage work has been deployed broadly at Meta. Code written with Folly’s coroutine library, [folly::coro], uses [folly::coro::AsyncScope] to safely launch awaitables. Most code written with Unifex, an implementation of an earlier version of the *Sender/Receiver* model proposed in [P2300R7], uses [unifex::v1::async_scope], although experience with the v1 design led to the creation of [unifex::v2::async_scope], which has a smaller interface and a cleaner definition of responsibility.

As an early adopter of Unifex, [rsys] (Meta’s cross-platform voip client library) became the entry point for structured concurrency in mobile code at Meta. We originally built rsys with an unstructured asynchrony model built around posting callbacks to threads in order to optimize for binary size. However, this came at the expense of developer velocity due to the increasing cost of debugging deadlocks and crashes resulting from race conditions.

We decided to adopt Unifex and refactor towards a more structured architecture to address these problems systematically. Converting an unstructured production codebase to a structured one is such a large project that it needs to be done in phases. As we began to convert callbacks to senders/tasks, we quickly realized that we needed a safe place to start structured asynchronous work in an unstructured environment. We addressed this need with `unifex::v1::async_scope` paired with an executor to address a recurring pattern:

Before	After
<pre> // Abstraction for thread that has the ability // to execute units of work. class Executor { public: virtual void add(Func function) noexcept = 0; }; // Example class class Foo { std::shared_ptr<Executor> exec_; public: void doSomething() { auto asyncWork = [&]() { // do something }; exec_>add(asyncWork); } }; </pre>	<pre> // Utility class for executing async work on an // async_scope and on the provided executor class ExecutorAsyncScopePair { unifex::v1::async_scope scope_; ExecutorScheduler exec_; public: void add(Func func) { scope_.detached_spawn_call_on(exec_, func); } auto cleanup() { return scope_.cleanup(); } }; // Example class class Foo { std::shared_ptr<ExecutorAsyncScopePair> exec_; public: ~Foo() { sync_wait(exec_>cleanup()); } void doSomething() { auto asyncWork = [&]() { // do something }; exec_>add(asyncWork); } }; </pre>

This broadly worked but we discovered that the above design coupled with the v1 API allowed for too many redundancies and conflated too many responsibilities (scoping async work, associating work with a stop source, and transferring scoped work to a new scheduler).

We learned that making each component own a distinct responsibility will minimize the confusion and increase the structured concurrency adoption rate. The above example was an intuitive use of `async_scope` because the concept of a “scoped executor” was familiar to many engineers and is a popular async pattern in other programming languages. However, the above design abstracted away some of the APIs in `async_scope` that explicitly asked for a scheduler, which would have helped challenge the assumption engineers made about `async_scope` being an instance of a “scoped executor”.

Cancellation was an unfamiliar topic for engineers within the context of asynchronous programming. The `v1::async_scope` provided both `cleanup()` and `complete()` to give engineers the freedom to decide between canceling work or waiting for work to finish. The different nuances on when this should happen and how it happens ended up being an obstacle that engineers didn't want to deal with.

Over time, we also found redundancies in the way `v1::async_scope` and other algorithms were implemented and identified other use cases that could benefit from a different kind of async scope. This motivated us to create `v2::async_scope` which only has one responsibility (scope), and `nest` which helped us improve maintainability and flexibility of Unifex.

The unstructured nature of `cleanup()/complete()` in a partially structured codebase introduced deadlocks when engineers nested the `cleanup()/complete()` sender in the scope being joined. This risk of deadlock remains with `v2::async_scope::join()` however, we do think this risk can be managed and is worth the tradeoff in exchange for a more coherent architecture that has fewer crashes. For example, we have experienced a significant reduction in these types of deadlocks once engineers understood that `join()` is a destructor-like operation that needs to be run only by the scope's owner. Since there is no language support to manage async lifetimes automatically, this insight was key in preventing these types of deadlocks. Although this breakthrough was a result of strong guidance from experts, we believe that the simpler design of `v2::async_scope` would make this a little easier.

We strongly believe that `async_scope` was necessary for making structured concurrency possible within `rsys`, and we believe that the improvements we made with `v2::async_scope` will make the adoption of P2300 more accessible.

§ 3 Motivation

§ 3.1 Motivating example

Let us assume the following code:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx; // create a global context for the application

    std::vector<work_item*> items = get_work_items();
    for (auto item : items) {
        // Spawn some work dynamically
        ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
            ex::then([&](work_item* item) { do_work(ctx, item); });
        ex::start_detached(std::move(snd));
    }
    // `ctx` and `my_pool` are destroyed
}
```

In this example we are creating parallel work based on the given input vector. All the work will be spawned on the local `static_thread_pool` object, and will use a shared `work_context` object.

Because the number of work items is dynamic, one is forced to use `start_detached()` from [P2300R7] (or something equivalent) to dynamically spawn work. [P2300R7] doesn't provide any facilities to spawn dynamic work and return a sender (i.e., something like `when_all` but with a dynamic number of input senders).

Using `start_detached()` here follows the *fire-and-forget* style, meaning that we have no control over, or awareness of, the completion of the async work that is being spawned.

At the end of the function, we are destroying the `work_context` and the `static_thread_pool`. But at that point, we don't know whether all the spawned async work has completed. If any of the async work is incomplete, this might lead to crashes.

[P2300R7] doesn't give us out-of-the-box facilities to use in solving these types of problems.

This paper proposes the `counting_scope` and [P3296R2]'s `let_async_scope` facilities that would help us avoid the invalid behavior. With `counting_scope`, one might write safe code this way:

```
namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();
```

```

int main() {
    static_thread_pool my_pool{8};
    work_context ctx;           // create a global context for the application
    ex::counting_scope scope; // create this *after* the resources it protects

    // make sure we always join
    unifex::scope_guard join = [&]() noexcept {
        // wait for all nested work to finish
        this_thread::sync_wait(scope.join()); // NEW!
    };

    std::vector<work_item*> items = get_work_items();
    for (auto item : items) {
        // Spawn some work dynamically
        ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
            ex::then([&](work_item* item) { do_work(ctx, item); });

        // start `snd` as before, but associate the spawned work with `scope` so that it can
        // be awaited before destroying the resources referenced by the work (i.e. `my_pool`
        // and `ctx`)
        ex::spawn(std::move(snd), scope.get_token()); // NEW!
    }

    // `ctx` and `my_pool` are destroyed *after* they are no longer referenced
}

```

With [P3296R2]’s `let_async_scope`, one might write safe code this way:

```

namespace ex = std::execution;

struct work_context;
struct work_item;
void do_work(work_context&, work_item*);
std::vector<work_item*> get_work_items();

int main() {
    static_thread_pool my_pool{8};
    work_context ctx; // create a global context for the application

    this_thread::sync_wait(
        ex::let_async_scope(ex::just(get_work_items()), [&](auto scope, auto& items) {
            for (auto item : items) {
                // Spawn some work dynamically
                ex::sender auto snd = ex::transfer_just(my_pool.get_scheduler(), item) |
                    ex::then([&](work_item* item) { do_work(ctx, item); });

                // start `snd` as before, but associate the spawned work with `scope` so that it
                // can be awaited before destroying the resources referenced by the work (i.e.
                // `my_pool` and `ctx`)
                ex::spawn(std::move(snd), scope); // NEW!
            }
            return just();
        }));

    // `ctx` and `my_pool` are destroyed *after* they are no longer referenced
}

```

Simplifying the above into something that fits in a Tony Table to highlight the differences gives us:

Before	With <code>counting_scope</code>	With <code>let_async_scope</code>
<pre> namespace ex = std::execution; struct context; ex::sender auto work(const context&); int main() { context ctx; ex::sender auto snd = work(ctx); // fire and forget ex::start_detached(std::move(snd)); // `ctx` is destroyed, perhaps // before // `snd` is done } </pre>	<pre> namespace ex = std::execution; struct context; ex::sender auto work(const context&); int main() { context ctx; ex::counting_scope scope; ex::sender auto snd = work(ctx); try { // fire, but don't forget ex::spawn(std::move(snd), scope.get_token()); } catch (...) { // do something to handle // exception } // wait for all work nested within // scope // to finish this_thread::sync_wait(scope.join()); // `ctx` is destroyed once nothing // references it } </pre>	<pre> namespace ex = std::execution; struct context; ex::sender auto work(const context&); int main() { context ctx; this_thread::sync_wait(ex::just() ex::let_async_scope([&] (auto scope) { ex::sender auto snd = work(ctx); // fire, but don't forget ex::spawn(std::move(snd), scope.get_token()); })); // `ctx` is destroyed once // nothing // references it } </pre>

Please see below for more examples.

§ 3.2 `counting_scope` and `let_async_scope` are a step forward towards Structured Concurrency

Structured Programming [Dahl72] transformed the software world by making it easier to reason about the code, and build large software from simpler constructs. We want to achieve the same effect on concurrent programming by ensuring that we *structure* our concurrent code. [P2300R7] makes a big step in that direction, but, by itself, it doesn't fully realize the principles of Structured Programming. More specifically, it doesn't always ensure that we can apply the *single entry, single exit point* principle.

The `start_detached` sender algorithm fails this principle by behaving like a `GOTO` instruction. By calling `start_detached` we essentially continue in two places: in the same function, and on different thread that executes the given work. Moreover, the lifetime of the work started by `start_detached` cannot be bound to the local context. This will prevent local reasoning, which will make the program harder to understand.

To properly structure our concurrency, we need an abstraction that ensures that all async work that is spawned has a defined, observable, and controllable lifetime. This is the goal of `counting_scope` and `let_async_scope`.

§ 4 Examples of use

§ 4.1 Spawning work from within a task

Use `let_async_scope` in combination with a `system_context` from [P2079R2] to spawn work from within a task:

```

namespace ex = std::execution;

int main() {
    ex::system_context ctx;
    int result = 0;

    ex::scheduler auto sch = ctx.scheduler();

    ex::sender auto val = ex::just() | ex::let_async_scope([sch](ex::async_scope_token auto scope) {
        int val = 13;

        auto print_sender = ex::just() | ex::then([val]() noexcept {
            std::cout << "Hello world! Have an int with value: " << val << "\n";
        });

        // spawn the print sender on sch
        //
    });
}

```

```

    // NOTE: if spawn throws, let_async_scope will capture the exception
    //         and propagate it through its set_error completion
    ex::spawn(ex::on(sch, std::move(print_sender)), scope);

    return ex::just(val);
} | ex::then([&result](auto val) { result = val });

this_thread::sync_wait(ex::on(sch, std::move(val)));

std::cout << "Result: " << result << "\n";
}

// 'let_async_scope' ensures that, if all work is completed successfully, the result will be 13
// 'sync_wait' will throw whatever exception is thrown by the callable passed to 'let_async_scope'

```

§ 4.2 Starting work nested within a framework

In this example we use the `counting_scope` within a class to start work when the object receives a message and to wait for that work to complete before closing.

```

namespace ex = std::execution;

struct my_window {
    class close_message {};

    ex::sender auto some_work(int message);

    ex::sender auto some_work(close_message message);

    void onMessage(int i) {
        ++count;
        ex::spawn(ex::on(sch, some_work(i)), scope);
    }

    void onClickClose() {
        ++count;
        ex::spawn(ex::on(sch, some_work(close_message{})), scope);
    }

    my_window(ex::system_scheduler sch, ex::counting_scope::token scope)
        : sch(sch)
        , scope(scope) {
        // register this window with the windowing framework somehow so that
        // it starts receiving calls to onClickClose() and onMessage()
    }

    ex::system_scheduler sch;
    ex::counting_scope::token scope;
    int count{0};
};

int main() {
    // keep track of all spawned work
    ex::counting_scope scope;
    ex::system_context ctx;
    try {
        my_window window{ctx.get_scheduler(), scope.get_token()};
    } catch (...) {
        // do something with exception
    }
    // wait for all work nested within scope to finish
    this_thread::sync_wait(scope.join());
    // all resources are now safe to destroy
    return window.count;
}

```

§ 4.3 Starting parallel work

In this example we use `let_async_scope` to construct an algorithm that performs parallel work. Here `foo` launches 100 tasks that concurrently run on some scheduler provided to `foo`, through its connected receiver, and then the tasks are asynchronously joined. This structure emulates how we might build a parallel algorithm where each `some_work` might be operating on a fragment of data.

```

namespace ex = std::execution;

ex::sender auto some_work(int work_index);

```

```

ex::sender auto foo(ex::scheduler auto sch) {
    return ex::just() | ex::let_async_scope([sch](ex::async_scope_token auto scope) {
        return ex::schedule(sch) | ex::then([] { std::cout << "Before tasks launch\n"; }) |
            ex::then([=] {
                // Create parallel work
                for (int i = 0; i < 100; ++i) {
                    // NOTE: if spawn() throws, the exception will be propagated as the
                    //       result of let_async_scope through its set_error completion
                    ex::spawn(ex::on(sch, some_work(i)), scope);
                }
            });
    }) | ex::then([] { std::cout << "After tasks complete successfully\n"; });
}

```

§ 4.4 Listener loop in an HTTP server

This example shows how one can write the listener loop in an HTTP server, with the help of coroutines. The HTTP server will continuously accept new connection and start work to handle the requests coming on the new connections. While the listening activity is bound in the scope of the loop, the lifetime of handling requests may exceed the scope of the loop. We use `counting_scope` to limit the lifetime of the request handling without blocking the acceptance of new requests.

```

namespace ex = std::execution;

task<size_t> listener(int port, io_context& ctx, static_thread_pool& pool) {
    size_t count{0};
    listening_socket listen_sock{port};

    co_await ex::let_async_scope(ex::just(), [&](ex::async_scope_token auto scope) -> task<void> {
        while (!ctx.is_stopped()) {
            // Accept a new connection
            connection conn = co_await async_accept(ctx, listen_sock);
            count++;

            // Create work to handle the connection in the scope of `work_scope`
            conn_data data{std::move(conn), ctx, pool};
            ex::sender auto snd = ex::just(std::move(data)) |
                ex::let_value([](auto& data) { return handle_connection(data); });

            ex::spawn(std::move(snd), scope);
        }
    });

    // At this point, all the request handling is complete
    co_return count;
}

```

[libunifex] has a very similar example HTTP server at [io_uring HTTP server] that compiles and runs on Linux-based machines with `io_uring` support.

§ 4.5 Pluggable functionality through composition

This example is based on real code in `rsys`, but it reduces the real code to slideware and ports it from Unifex to the proposed `std::execution` equivalents. The central abstraction in `rsys` is a `Call`, but each integration of `rsys` has different needs so the set of features supported by a `Call` varies with the build configuration. We support this configurability by exposing the equivalent of the following method on the `Call` class:

```

template <typename Feature>
Handle<Feature> Call::get();

```

and it's used like this in app-layer code:

```

unifex::task<void> maybeToggleCamera(Call& call) {
    Handle<Camera> camera = call.get<Camera>();

    if (camera) {
        co_await camera->toggle();
    }
}

```

A `Handle<Feature>` is effectively a part-owner of the `Call` it came from.

The team that maintains `rsys` and the teams that use `rsys` are, unsurprisingly, different teams so `rsys` has to be designed to solve organizational problems as well as technical problems. One relevant design decision the `rsys` team made is that it is safe to keep using a `Handle<Feature>` after the end of its `Call`'s lifetime; this choice adds some complexity to the design of `Call` and its various features but

it also simplifies the support relationship between the rsys team and its many partner teams because it eliminates many crash-at-shutdown bugs.

```
namespace rsys {

class Call {
public:
    unifex::nothrow_task<void> destroy() noexcept {
        // first, close the scope to new work and wait for existing work to finish
        scope_>close();
        co_await scope_>join();

        // other clean-up tasks here
    }

    template <typename Feature>
    Handle<Feature> get() noexcept;

private:
    // an async scope shared between a call and its features
    std::shared_ptr<std::execution::counting_scope> scope_;
    // each call has its own set of threads
    ExecutionContext context_;

    // the set of features this call supports
    FeatureBag features_;
};

class Camera {
public:
    std::execution::sender auto toggle() {
        namespace ex = std::execution;

        return ex::just() | ex::let_value([this]() {
            // this callable is only invoked if the Call's scope is in
            // the open or unused state when nest() is invoked, making
            // it safe to assume here that:
            //
            // - scheduler_ is not a dangling reference to the call's
            //   execution context
            // - Call::destroy() has not progressed past starting the
            //   join-sender so all the resources owned by the call
            //   are still valid
            //
            // if the nest() attempt fails because the join-sender has
            // started (or even if the Call has been completely destroyed)
            // then the sender returned from toggle() will safely do
            // nothing before completing with set_stopped()

            return ex::schedule(scheduler_) | ex::then([this]() {
                // toggle the camera
            });
        }) | ex::nest(callScope_>get_token());
    }

private:
    // a copy of this camera's Call's scope_ member
    std::shared_ptr<ex::counting_scope> callScope_;
    // a scheduler that refers to this camera's Call's ExecutionContext
    Scheduler scheduler_;
};

} // namespace rsys
```

§ 4.6 Recursively spawning work until completion

Below are three ways you could recursively spawn work on a scope using `let_async_scope` or `counting_scope`.

§ 4.6.1 `let_async_scope` with `spawn()`

```
struct tree {
    std::unique_ptr<tree> left;
    std::unique_ptr<tree> right;
    int data;
};

auto process(ex::scheduler auto sch, auto scope, tree& t) noexcept {
```

```

    return ex::schedule(sch) | then([sch, &]() {
        if (t.left)
            ex::spawn(process(sch, scope, t.left.get()), scope);
        if (t.right)
            ex::spawn(process(sch, scope, t.right.get()), scope);
        do_stuff(t.data);
    }) | ex::let_error([](auto& e) {
        // log error
        return just();
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    // let_async_scope will ensure all new work will be spawned on the
    // scope and will not be joined until all work is finished.
    // NOTE: Exceptions will not be surfaced to let_async_scope; exceptions
    // will be handled by let_error instead.
    this_thread::sync_wait(ex::just() | ex::let_async_scope([&, sch](auto scope) {
        return process(sch, scope, t);
    }));
}

```

§ 4.6.2 let_async_scope with spawn_future()

```

struct tree {
    std::unique_ptr<tree> left;
    std::unique_ptr<tree> right;
    int data;
};

auto process(ex::scheduler auto sch, auto scope, tree& t) {
    return ex::schedule(sch) | ex::let_value([sch, &]() {
        unifex::any_sender_of<> leftFut = ex::just();
        unifex::any_sender_of<> rightFut = ex::just();
        if (t.left) {
            leftFut = ex::spawn_future(process(sch, scope, t.left.get()), scope);
        }

        if (t.right) {
            rightFut = ex::spawn_future(process(sch, scope, t.right.get()), scope);
        }

        do_stuff(t.data);
        return ex::when_all(leftFut, rightFut) | ex::then([](auto&&...) noexcept {});
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    // let_async_scope will ensure all new work will be spawned on the
    // scope and will not be joined until all work is finished
    // NOTE: Exceptions will be surfaced to let_async_scope which will
    // call set_error with the exception_ptr
    this_thread::sync_wait(ex::just() | ex::let_async_scope([&, sch](auto scope) {
        return process(sch, scope, t);
    }));
}

```

§ 4.6.3 counting_scope

```

struct tree {
    std::unique_ptr<tree> left;
    std::unique_ptr<tree> right;
    int data;
};

auto process(ex::counting_scope_token scope, ex::scheduler auto sch, tree& t) noexcept {
    return ex::schedule(sch) | ex::then([sch, &]() noexcept {
        if (t.left)
            ex::spawn(process(scope, sch, t.left.get()), scope);

        if (t.right)
            ex::spawn(process(scope, sch, t.right.get()), scope);

        do_stuff(t.data);
    });
}

```

```

    }) | ex::let_error([](auto& e) {
        // log error
        return just();
    });
}

int main() {
    ex::scheduler sch;
    tree t = make_tree();
    ex::counting_scope scope;
    ex::spawn(process(scope.get_token(), sch, t), scope.get_token());
    this_thread::sync_wait(scope.join());
}

```

§ 5 Async Scope, usage guide

An async scope is a type that implements a “bookkeeping policy” for senders that have been associated with the scope. Depending on the policy, different guarantees can be provided in terms of the lifetimes of the scope and any associated senders. The `counting_scope` described in this paper defines a policy that has proven useful while progressively adding structure to existing, unstructured code at Meta, but other useful policies are possible. By defining `nest()`, `spawn()`, and `spawn_future()` in terms of the more fundamental async scope token interface, and leaving the implementation of the abstract interface to concrete token types, this paper’s design leaves the set of policies open to extension by user code or future standards.

An async scope token’s implementation of the `async_scope_token` concept:

- must allow an arbitrary sender to be wrapped without eagerly starting the sender;
- must not add new value or error completions when wrapping a sender;
- may fail to associate a new sender by returning `false` from `try_associate()`;
- may fail to associate a new sender by eagerly throwing an exception from either `try_associate()` or `wrap()`;

More on these items can be found below in the sections below.

§ 5.1 Definitions

```

namespace std::execution {

struct spawn_receiver { // exposition-only
    void set_value() && noexcept;
    void set_stopped() && noexcept;
};

template <class Sigs>
struct spawn_future_receiver { // exposition-only
    template <class... T>
    void set_value(T&&... t) && noexcept;

    template <class E>
    void set_error(E&& e) && noexcept;

    void set_stopped() && noexcept;
};

template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> same_as<bool>;
        { token.disassociate() } -> same_as<void>;
    };

template <async_scope_token Token, sender Sender>
using wrapped_sender_from = decay_t<decltype(declval<Token&>().wrap(declval<Sender>()))>; // exposition-only

struct nest_t { unspecified };
struct spawn_t { unspecified };
struct spawn_future_t { unspecified };

inline constexpr nest_t nest{};
inline constexpr spawn_t spawn{};
inline constexpr spawn_future_t spawn_future{};

class simple_counting_scope {
    struct token {

```

```

    template <sender Sender>
    Sender&& wrap(Sender&& snd) const noexcept;

    bool try_associate() const;

    void disassociate() const;

private:
    simple_counting_scope* scope; // exposition-only
};

simple_counting_scope() noexcept;
~simple_counting_scope();

// simple_counting_scope is immovable and uncopyable
simple_counting_scope(simple_counting_scope&&) = delete;

token get_token() noexcept;

void close() noexcept;

sender auto join() noexcept;
};

class counting_scope {
    struct token {
        template <sender Sender>
        sender auto wrap(Sender&& snd) const;

        bool try_associate() const;

        void disassociate() const;

private:
        counting_scope* scope; // exposition-only
    };

    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(counting_scope&&) = delete;

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    sender auto join() noexcept;
};

} // namespace std::execution

```

§ 5.2 execution::async_scope_token

```

template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> same_as<bool>;
        { token.disassociate() } -> same_as<void>;
    };

```

An async scope token is a non-owning handle to an async scope that behaves like a reference-to-async-scope; tokens are no-throw copyable and movable, and it is undefined behaviour to invoke any methods on a token that has outlived its scope.

The `try_associate()` method on a token attempts to create a new association with the scope; `try_associate()` returns true when the association is successful, and it may either return false or throw an exception to indicate failure. Returning false will generally lead to algorithms that operate on tokens behaving as if provided a sender that completes immediately with `set_stopped()`, leading to rejected work being discarded as a “no-op”. Throwing an exception will generally lead to that exception escaping from the calling algorithm.

The `disassociate()` method removes a previously-established association with the scope. `disassociate()` must be called exactly once for every call to `try_associate()` that returns true; it is undefined behaviour to do otherwise.

Tokens also have a `wrap()` method that takes and returns a sender. The `wrap()` method gives the token an opportunity to modify the input sender's behaviour in a scope-specific way. The proposed `counting_scope` uses this opportunity to associate the input sender with a stop token that the scope can use to request stop on all outstanding operations associated within the scope.

In order to provide the Strong Exception Guarantee, the algorithms proposed in this paper invoke `token.wrap(snd)` before invoking `token.try_associate()`. Other algorithms written in terms of `async_scope_token` should do the same.

Note: Wrapping the sender before creating an association means that, when `try_associate()` returns `false`, the work to wrap the sender must be discarded. We could, instead, try to create the association first and only wrap the sender when successful; this would be more efficient but would limit us to providing the Basic Exception Guarantee.

§ 5.3 `execution::nest`

```
struct nest_t { unspecified };  
inline constexpr nest_t nest{};
```

`nest` is a CPO with the following signature:

```
sender auto nest(sender auto&&, async_scope_token auto) noexcept(...);
```

When successful, `nest()` creates an association with the given token's scope and returns an “associated” nest-sender that behaves the same as its input sender, with the following additional effects:

- the association ends when the nest-sender is destroyed or, if it is connected, when the resulting operation state is destroyed; and
- whatever effects are added by the token's `wrap()` method.

When unsuccessful, `nest()` will either return an “unassociated” nest-sender or it will allow any thrown exceptions to escape.

When `nest()` returns an associated nest-sender:

- connecting and starting the associated nest-sender connects and starts the given sender; and
- the associated nest-sender has exactly the same completions as the input sender.

When `nest()` returns an unassociated nest-sender:

- the input sender is discarded and will never be connected or started; and
- the unassociated nest-sender will only complete with `set_stopped()`.

Given an `async_scope_token`, `token`, and a sender, `snd`, `nest(snd, token)` is expression-equivalent to `make_sender(nest, nest_data{snd, token})`, where `nest_data` is an exposition-only class whose constructor performs the following operations in the following order:

1. store the result of `token.wrap(snd)` in a member variable
2. invoke `token.try_associate()`
 - a. if the result is `false` then destroy the previously stored result of `token.wrap(snd)`; the nest-sender under construction is unassociated.
 - b. otherwise, the nest-sender under construction is associated.

Any exceptions thrown during the evaluation of the constructor are allowed to escape; nevertheless, `nest()` provides the Strong Exception Guarantee.

An associated nest-sender has many properties of an RAII handle:

- constructing an instance acquires a “resource” (the association with the scope)
- destructing an instance releases the same resource
- moving an instance into another transfers ownership of the resource from the source to the destination
- etc.

Copying a nest-sender is possible if the sender it is wrapping is copyable but the copying process is a bit unusual because of the `async_scope_association` it contains. If the sender, `snd`, provided to `nest()` is copyable then the resulting nest-sender is also copyable, with the following rules:

- copying an unassociated nest-sender invariably produces a new unassociated nest-sender; and
- copying an associated nest-sender requires copying the `nest_data` it contains and the `nest_data` copy-constructor proceeds as follows:

1. copy the token from the source *nest-data* to the destination; and
2. if `token.try_associate()` returns `true`, copy the sender from the source *nest-data* to the destination
 - if the sender is copied in step 2 then the new *nest-sender* is associated; otherwise, it's unassociated

Note: copying an associated *nest-sender* may produce an unassociated *nest-sender* however this observable difference is not a salient property of the *nest-sender*. A *nest-sender* is similar to a stateful `std::function<T()>` for some `T`; it is expected that invoking a copy of such an object may produce a different result than invoking the original.

When a *nest-sender* has a copy constructor, it provides the Strong Exception Guarantee.

When connecting an unassociated *nest-sender*, the resulting *operation-state* completes immediately with `set_stopped()` when started.

When connecting an associated *nest-sender*, there are four possible outcomes:

1. the *nest-sender* is rvalue connected, which infallibly moves the sender's association from the sender to the *operation-state*
2. the *nest-sender* is lvalue connected, in which case the new *operation-state* needs its own association with the *nest-sender*'s scope, which requires calling `try_associate()` again and it may:
 - a. succeed by returning `true`;
 - b. fail by returning `false`, in which case the *operation-state* behaves as if it were constructed from an unassociated *nest-sender*; or
 - c. fail by throwing an exception, in which case the exception escapes from the call to connect.

An *operation-state* with its own association must invoke `token.disassociate()` as the last step of the *operation-state*'s destructor.

Note: the timing of when an associated *operation-state* ends its association with the scope is chosen to avoid exposing user code to dangling references. Scopes are expected to serve as mechanisms for signaling when it is safe to destroy shared resources being protected by the scope. Ending any given association with a scope may lead to that scope signaling that the protected resources can be destroyed so a *nest-sender*'s *operation-state* must not permit that signal to be sent until the *operation-state* is definitely finished accessing the shared resources, which is at the end of the *operation-state*'s destructor.

A call to `nest()` does not start the given sender and is not expected to incur allocations.

Regardless of whether the returned sender is associated or unassociated, it is multi-shot if the input sender is multi-shot and single-shot otherwise.

§ 5.4 `execution::spawn`

```
template <class Env>
struct spawn_receiver { // exposition-only
    void set_value() noexcept;
    void set_stopped() noexcept;
};

struct spawn_t { unspecified };

inline constexpr spawn_t spawn{};
```

`spawn` is a CPO with the following signature:

```
template <sender Sender, async_scope_token Token, class Env = empty_env>
void spawn(Sender&& snd, Token token, Env env = {});
```

`spawn` attempts to associate the given sender with the given scope token's scope. On success, the given sender is eagerly started. On failure, either the sender is discarded and no further work happens or `spawn()` throws.

Starting the given sender without waiting for it to finish requires a dynamic allocation of the sender's *operation-state*. The following algorithm determines which *Allocator* to use for this allocation:

- If `get_allocator(env)` is valid and returns an *Allocator* then choose that *Allocator*.
- Otherwise, if `get_allocator(get_env(token.wrap(snd)))` is valid and returns an *Allocator* then choose that *Allocator*.
- Otherwise, choose `std::allocator<void>`.

`spawn()` proceeds with the following steps in the following order:

1. an environment, `senv`, is chosen:
 - if `get_allocator(env)` is valid then `senv` is `env`;

- otherwise, if `get_allocator(get_env(token.wrap(snd)))` is valid then `senv` is the expression `JOIN-ENV(env, MAKE-ENV(get_allocator, alloc))`, where `alloc` is the chosen allocator;
 - otherwise, `senv` is `env`.
2. the type of the object to dynamically allocate is computed, say `op_t`; `op_t` contains
 - an *operation-state*;
 - an allocator of the chosen type; and
 - a copy of `token`.
 3. an `op_t` is dynamically allocated by the *Allocator* chosen as described above
 4. the fields of the `op_t` are initialized in the following order:
 - a. the *operation-state* within the allocated `op_t` is initialized with the result of `connect(write-env(token.wrap(std::forward<Sender>(snd)), spawn-receiver{...}, senv))`;
 - b. the allocator is initialized with a copy of the allocator used to allocate the `op_t`; and
 - c. the token is initialized with a copy of `token`.
 5. if `token.try_associate()` returns true then the *operation-state* is started; otherwise, the `op_t` is destroyed and deallocated.

Any exceptions thrown during the execution of `spawn()` are allowed to escape; nevertheless, `spawn()` provides the Strong Exception Guarantee.

Upon completion of the *operation-state*, the *spawn-receiver* performs the following steps:

1. move the allocator and token from the `op_t` into local variables;
2. destroy the *operation-state*;
3. use the local copy of the allocator to deallocate the `op_t`;
4. destroy the local copy of the allocator; and
5. invoke `token.disassociate()` on the local copy of the token.

Performing step 5 last ensures that all possible references to resources protected by the scope, including possibly the allocator, are no longer in use before dissociating from the scope.

This is similar to `start_detached()` from [P2300R7], but the scope may observe and participate in the lifetime of the work described by the sender. The `simple_counting_scope` and `counting_scope` described in this paper use this opportunity to keep a count of spawned senders that haven't finished, and to prevent new senders from being spawned once the scope has been closed.

The given sender must complete with `set_value()` or `set_stopped()` and may not complete with an error; the user must explicitly handle the errors that might appear as part of the *sender-expression* passed to `spawn()`.

User expectations will be that `spawn()` is asynchronous and so, to uphold the principle of least surprise, `spawn()` should only be given non-blocking senders. Using `spawn()` with a sender generated by `on(sched, blocking-sender)` is a very useful pattern in this context.

NOTE: A query for non-blocking start will allow `spawn()` to be constrained to require non-blocking start.

Usage example:

```
...
for (int i = 0; i < 100; i++)
    spawn(on(sched, some_work(i)), scope.get_token());
```

§ 5.5 execution::spawn_future

```
template <class Sigs>
struct spawn_future_receiver { // exposition-only
    template <class... T>
    void set_value(T&&... t) && noexcept;

    template <class E>
    void set_error(E&& e) && noexcept;

    void set_stopped() && noexcept;
};

struct spawn_future_t { unspecified };

inline constexpr spawn_future_t spawn_future{};
```

spawn_future is a CPO with the following signature:

```
template <sender Sender, async_scope_token Token, class Env = empty_env>
sender auto spawn_future(Sender&& snd, Token token, Env env = {});
```

spawn_future attempts to associate the given sender with the given scope token's scope. On success, the given sender is eagerly started and spawn_future returns a sender that provides access to the result of the given sender. On failure, either spawn_future returns a sender that unconditionally completes with set_stopped() or it throws.

Similar to spawn(), starting the given sender involves a dynamic allocation of some state. spawn_future() chooses an *Allocator* for this allocation in the same way spawn() does: use the result of get_allocator(env) if that is a valid expression, otherwise use the result of get_allocator(get_env(token.wrap(snd))) if that is a valid expression, otherwise use a std::allocator<void>.

Compared to spawn(), the dynamically allocated state is more complicated because it must contain storage for the result of the given sender, however it eventually completes, and synchronization facilities for resolving the race between the given sender's production of its result and the returned sender's consumption or abandonment of that result.

Unlike spawn(), spawn_future() returns a sender rather than void. The returned sender, fs, is a handle to the spawned work that can be used to consume or abandon the result of that work. The completion signatures of fs include set_stopped() and all the completion signatures of the spawned sender. When fs is connected and started, it waits for the spawned sender to complete and then completes itself with the spawned sender's result.

spawn_future(snd, token, env) proceeds with the following steps in the following order:

1. An allocator, alloc, is chosen as described above.
2. A stop token, stok, is chosen as follows:
 - if get_stop_token(env) is a well-defined then stok is a stop token that receives stop requests sent by the returned future *and* any stop requests received by the stop token returned from get_stop_token(env);
 - otherwise, stok is a stop token that receives stop requests sent by the returned future.
3. An environment, senv, is chosen as follows:
 - if alloc is get_allocator(env) then senv is JOIN-ENV(env, MAKE-ENV(get_stop_token, stok));
 - otherwise, if alloc is get_allocator(get_env(token.wrap(snd))) then senv is JOIN-ENV(env, MAKE-ENV(get_allocator, alloc), MAKE-ENV(get_stop_token, stok));
 - otherwise, senv is JOIN-ENV(env, @MAKE-ENV(get_stop_token, stok)).
4. Storage for the spawned sender's state is dynamically allocated using alloc; the address of this storage is known as op.
5. The state for the spawned sender is constructed in the allocated storage
 - A subset of this state is an *operation-state* created with the following expression:

```
connect(
    write-env(token.wrap(snd), senv),
    spawn-future-receiver<completion-signatures-of<Sender>>{op}));
```

- After the last field in the dynamically allocated state is initialized,
 - If token.try_associate() returns true then the *operation-state* within the allocated state is started.
 - Otherwise the dynamically-allocated state is marked as having completed with set_stopped().
- 6. A sender is returned that, when connected and started, will complete with the result of the eagerly-started work.

Any exceptions thrown during the execution of spawn_future() are allowed to escape; nevertheless, spawn_future() provides the Strong Exception Guarantee.

Given a sender returned from spawn_future(), fs, if fs is destroyed without being connected, or if it is connected and the resulting *operation-state*, fsop, is destroyed without being started, then the eagerly-started work is “abandoned”.

Abandoning the eagerly-started work means:

- a stop request is sent to the running *operation-state*;
- any result produced by the running *operation-state* is discarded when the operation completes; and
- after the operation completes, the dynamically-allocated state is “cleaned up”.

Cleaning up the dynamically-allocated state means doing the following, in order:

1. the allocator and token in the state are moved into local variables;

2. the state is destroyed;
3. the dynamic allocation is deallocated with the local copy of the allocator;
4. the local copy of the allocator is destroyed; and
5. `token.disassociate()` is invoked on the local copy of the token.

When `fsop` is started, if `fsop` receives a stop request from its receiver before the eagerly-started work has completed then an attempt is made to abandon the eagerly-started work. Note that it's possible for the eagerly-started work to complete while `fsop` is requesting stop; once the stop request has been delivered, either `fsop` completes with the result of the eagerly-started work if it's ready, or it completes with `set_stopped()` without waiting for the eagerly-started work to complete.

When `fsop` is started and does not receive a stop request from its receiver, `fsop` completes after the eagerly-started work completes with the same completion. Once `fsop` completes, it cleans up the dynamically-allocated state.

`spawn_future` is similar to `ensure_started()` from [P2300R7], but the scope may observe and participate in the lifetime of the work described by the sender. The `simple_counting_scope` and `counting_scope` described in this paper use this opportunity to keep a count of given senders that haven't finished, and to prevent new senders from being started once the scope has been closed.

Unlike `spawn()`, the sender given to `spawn_future()` is not constrained on a given shape. It may send different types of values, and it can complete with errors.

Usage example:

```
...
sender auto snd = spawn_future(on(sched, key_work()), token) | then(continue_fun);
for (int i = 0; i < 10; i++)
    spawn(on(sched, other_work(i)), token);
return when_all(scope.join(), std::move(snd));
```

§ 5.6 execution::simple_counting_scope

```
class simple_counting_scope {
    struct token {
        template <sender Sender>
        Sender&& wrap(Sender&& snd) const noexcept;

        bool try_associate() const;

        void disassociate() const;

    private:
        simple_counting_scope* scope; // exposition-only
    };

    simple_counting_scope() noexcept;
    ~simple_counting_scope();

    // simple_counting_scope is immovable and uncopyable
    simple_counting_scope(simple_counting_scope&&) = delete;

    token get_token() noexcept;

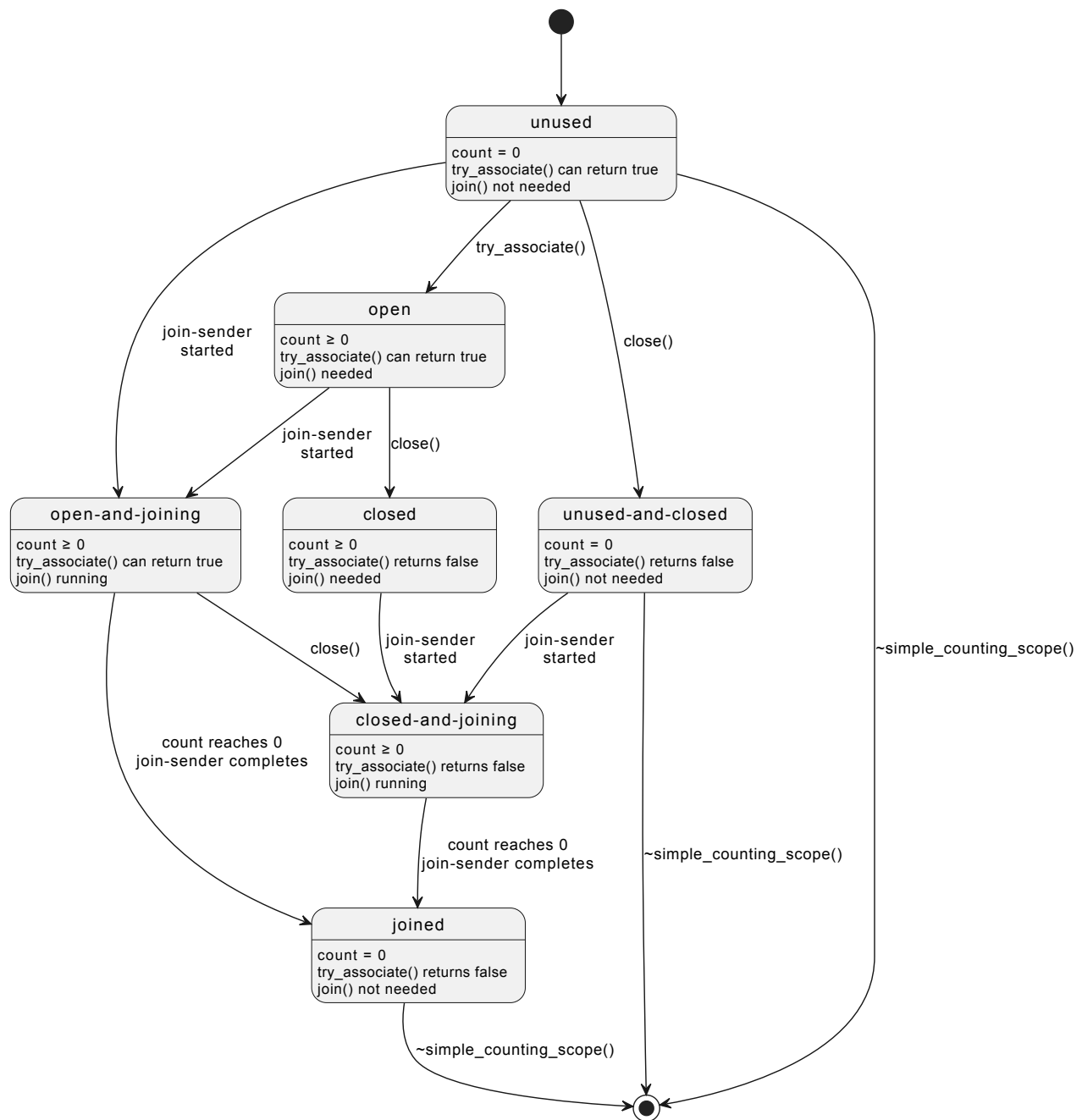
    void close() noexcept;

    sender auto join() noexcept;
};
```

A `simple_counting_scope` maintains a count of outstanding operations and goes through several states during its lifetime:

- unused
- open
- closed
- open-and-joining
- closed-and-joining
- unused-and-closed
- joined

The following diagram illustrates the `simple_counting_scope`'s state machine:



Note: a scope is “open” if its current state is unused, open, or open-and-joining; a scope is “closed” if its current state is closed, unused-and-closed, closed-and-joining, or joined.

Instances start in the unused state after being constructed. This is the only time the scope’s state can be set to unused. When the `simple_counting_scope` destructor starts, the scope must be in the unused, unused-and-closed, or joined state; otherwise, the destructor invokes `std::terminate()`. Permitting destruction when the scope is in the unused or unused-and-closed state ensures that instances of `simple_counting_scope` can be used safely as data-members while preserving structured functionality.

Connecting and starting a join-sender returned from `join()` moves the scope to either the open-and-joining or closed-and-joining state. Merely calling `join()` or connecting the join-sender does not change the scope’s state—the *operation-state* must be started to effect the state change. A started join-sender completes when the scope’s count of outstanding operations reaches zero, at which point the scope transitions to the joined state.

Calling `close()` on a `simple_counting_scope` moves the scope to the closed, unused-and-closed, or closed-and-joining state, and causes all future calls to `try_associate()` to return false.

Associating work with a `simple_counting_scope` can be done through `simple_counting_scope`’s token, which provides three methods: `wrap(sender auto&& s)`, `try_associate()`, and `disassociate()`.

— `wrap(sender auto&& s)` takes in a sender and returns it unmodified.

— `try_associate()` attempts to create a new association with the `simple_counting_scope` and will return `true` when successful, or `false`. The requirements for `try_associate()`'s success are outlined below:

1. While a scope is in the unused, open, or open-and-joining state, calls to `token.try_associate()` succeeds by incrementing the scope's count of outstanding operations before returning `true`.
2. While a scope is in the closed, unused-and-closed, closed-and-joining, or joined state, calls to `token.try_associate()` will return `false` and *will not* increment the scope's count of outstanding operations.

When a token's `try_associate()` returns `true`, the caller is responsible for undoing the association by invoking `disassociate()`, which will decrement the scope's count of outstanding operations.

— When a scope is in the open-and-joining or closed-and-joining state and a call to `disassociate()` undoes the final scope association, the scope moves to the joined state and the outstanding join-sender completes.

The state transitions of a `simple_counting_scope` mean that it can be used to protect asynchronous work from use-after-free errors. Given a resource, `res`, and a `simple_counting_scope`, `scope`, obeying the following policy is enough to ensure that there are no attempts to use `res` after its lifetime ends:

- all senders that refer to `res` are associated with `scope`; and
- `scope` is destroyed (and therefore in the joined, unused, or unused-and-closed state) before `res` is destroyed.

It is safe to destroy a scope in the unused or unused-and-closed state because there can't be any work referring to the resources protected by the scope.

A `simple_counting_scope` is uncopyable and immovable so its copy and move operators are explicitly deleted. `simple_counting_scope` could be made movable but it would cost an allocation so this is not proposed.

§ 5.6.1 `simple_counting_scope::simple_counting_scope`

```
simple_counting_scope() noexcept;
```

Initializes a `simple_counting_scope` in the unused state with the count of outstanding operations set to zero.

§ 5.6.2 `simple_counting_scope::~simple_counting_scope`

```
~simple_counting_scope();
```

Checks that the `simple_counting_scope` is in the joined, unused, or unused-and-closed state and invokes `std::terminate()` if not.

§ 5.6.3 `simple_counting_scope::get_token`

```
simple_counting_scope::token get_token() noexcept;
```

Returns a `simple_counting_scope::token` with `scope` set to this.

§ 5.6.4 `simple_counting_scope::close`

```
void close() noexcept;
```

Moves the scope to the closed, unused-and-closed, or closed-and-joining state. After a call to `close()`, all future calls to `try_associate()` return `false`.

§ 5.6.5 `simple_counting_scope::join`

```
sender auto join() noexcept;
```

Returns a join-sender. When the join-sender is connected to a receiver, `r`, it produces an *operation-state*, `o`. When `o` is started, the scope moves to either the open-and-joining or closed-and-joining state. `o` completes with `set_value()` when the scope moves to the joined state, which happens when the scope's count of outstanding operations drops to zero. `o` may complete synchronously if it happens to observe that the count of outstanding operations is already zero when started; otherwise, `o` completes on the execution context associated with the scheduler in its receiver's environment by asking its receiver, `r`, for a scheduler, `sch`, with `get_scheduler(get_env(r))` and then starting the sender returned from `schedule(sch)`. This requirement to complete on the receiver's scheduler restricts which receivers a join-sender may be connected to in exchange for determinism; the alternative would have the join-sender completing on the execution context of whichever nested operation happens to be the last one to complete.

§ 5.6.6 `simple_counting_scope::token::wrap`

```
template <sender Sender>
Sender&& wrap(Sender&& s) const noexcept;
```

Returns the argument unmodified.

§ 5.6.7 `simple_counting_scope::token::try_associate`

```
bool try_associate() const;
```

The following atomic state change is attempted on the token's scope:

- increment the scope's count of outstanding operations; and
- move the scope to the open state if it was in the unused state.

The atomic state change succeeds and the method returns `true` if the scope is observed to be in the unused, open, or open-and-joining state; otherwise the scope's state is left unchanged and the method returns `false`.

§ 5.6.8 `simple_counting_scope::token::disassociate`

```
void disassociate() const;
```

Decrements the associated scope's count of outstanding operations and, when the scope is in the open-and-joining or closed-and-joining state, moves the scope to the joined state and signals the outstanding join-sender to complete.

§ 5.7 `execution::counting_scope`

```
class counting_scope {
    struct token {
        template <sender Sender>
        sender auto wrap(Sender&& snd);

        bool try_associate() const;

        void disassociate() const;

    private:
        counting_scope* scope; // exposition-only
    };

    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(counting_scope&&) = delete;

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    sender auto join() noexcept;
};
```

A `counting_scope` behaves like a `simple_counting_scope` augmented with a stop source; the `wrap` method on a `counting_scope`'s token returns a sender that, when connected and started, produces an *operation-state* that receives stop requests from both its receiver and from the `counting_scope`. This extension of `simple_counting_scope` allows a `counting_scope` to request stop on all of its outstanding operations by requesting stop on its stop source.

Assuming an exposition-only `stop_when(sender auto&&, stoppable_token auto)` (explained below), `counting_scope` behaves as if it were implemented like so:

```
class counting_scope {
    struct token {
        template <sender S>
        sender auto wrap(S&& snd) const
            noexcept(std::is_nothrow_constructible_v<std::remove_cvref_t<S>, S>) {
            return stop_when(std::forward<S>(snd), scope->source_.get_token());
        }

        async_scope_association auto try_associate() const {
            return scope->scope_.get_token().try_associate();
        }

    private:
        friend counting_scope;

        explicit token(counting_scope* scope) noexcept
            : scope_(scope) {}
    };

    counting_scope() noexcept;
    ~counting_scope();

    // counting_scope is immovable and uncopyable
    counting_scope(counting_scope&&) = delete;

    token get_token() noexcept;

    void close() noexcept;

    void request_stop() noexcept;

    sender auto join() noexcept;
};
```

```

        counting_scope* scope_;
    };

    token get_token() noexcept { return token{this}; }

    void close() noexcept { return scope_.close(); }

    void request_stop() noexcept { source_.request_stop(); }

    sender auto join() noexcept { return scope_.join(); }

private:
    simple_counting_scope scope_;
    inplace_stop_source source_;
};

```

`stop_when(sender auto&& snd, stoppable_token auto token)` is an exposition-only sender algorithm that maps its input sender, `snd`, to an output sender, `osnd`, such that, when `osnd` is connected to a receiver, `r`, the resulting *operation-state* behaves the same as connecting the original sender, `snd`, to `r`, except that `snd` will receive a stop request when either the token returned from `get_stop_token(r)` receives a stop request or when `token` receives a stop request.

Other than the use of `stop_when()` in `counting_scope::token::wrap()` and the addition of `request_stop()` to the interface, `counting_scope` has the same behavior and lifecycle as `simple_counting_scope`.

§ 5.7.1 `counting_scope::counting_scope`

```
counting_scope() noexcept;
```

Initializes a `counting_scope` in the unused state with the count of outstanding operations set to zero.

§ 5.7.2 `counting_scope::~counting_scope`

```
~counting_scope();
```

Checks that the `counting_scope` is in the joined, unused, or unused-and-closed state and invokes `std::terminate()` if not.

§ 5.7.3 `counting_scope::get_token`

```
counting_scope::token get_token() noexcept;
```

Returns a `counting_scope::token` with `scope` set to `this`.

§ 5.7.4 `counting_scope::close`

```
void close() noexcept;
```

Moves the scope to the closed, unused-and-closed, or closed-and-joining state. After a call to `close()`, all future calls to `try_associate()` return disengaged associations.

§ 5.7.5 `counting_scope::request_stop`

```
void request_stop() noexcept;
```

Requests stop on the scope's internal stop source. Since all senders associated with the scope have been given stop tokens from this internal stop source, the effect is to send stop requests to all outstanding (and future) nested operations.

§ 5.7.6 `counting_scope::join`

```

struct join_sender; // exposition-only

join_sender join() noexcept;

```

Returns a join-sender that behaves the same as the result of `simple_counting_scope::join()`. Connecting and starting the join-sender moves the scope to the open-and-joining or closed-and-joining state; the join-sender completes when the scope's count of outstanding operations drops to zero, at which point the scope moves to the joined state.

§ 5.7.7 `counting_scope::token::wrap`

```

template <sender Sender>
sender auto wrap(Sender&& snd);

```

Returns a sender, `osnd`, that behaves in all ways the same as the input sender, `snd`, except that, when `osnd` is connected to a receiver, the resulting *operation-state* receives stop requests from *both* the connected receiver *and* the stop source in the token's `counting_scope`.

§ 5.7.8 `counting_scope::token::try_associate`

```
bool try_associate() const;
```

Returns true if the token's scope is open, and false if it's closed. `try_associate()` behaves as if its `counting_scope` owns a `simple_counting_scope`, `scope`, and the result is equivalent to the result of invoking `scope.get_token().try_associate()`.

§ 5.8 When to use `counting_scope` vs [\[P3296R2\]](#)'s `let_async_scope`

Although `counting_scope` and `let_async_scope` have overlapping use-cases, we specifically designed the two facilities to address separate problems. In short, `counting_scope` is best used in an unstructured context and `let_async_scope` is best used in a structured context.

We define “unstructured context” as:

- a place where using `sync_wait` would be inappropriate,
- and you can't “solve by induction” (i.e you're not in an `async` context where you can start the sender by “awaiting” it)

`counting_scope` should be used when you have a sender you want to start in an unstructured context. In this case, `spawn(sender, scope.get_token())` would be the preferred way of starting asynchronous work. `scope.join()` needs to be called before the owning object's destruction in order to ensure that the object's lifetime lives at least until all asynchronous work completes. Note that exception safety needs to be handled explicitly in the use of `counting_scope`.

`let_async_scope` returns a sender, and therefore can only be started in one of 3 ways:

1. `sync_wait`
2. `spawn` on a `counting_scope`
3. `co_await`

`let_async_scope` will manage the scope for you, ensuring that the managed scope is always joined before `let_async_scope` completes. The algorithm frees the user from having to manage the coupling between the lifetimes of the managed scope and the resource(s) it protects with the limitation that the nested work must be fully structured. This behavior is a feature, since the scope being managed by `let_async_scope` is intended to live only until the sender completes. This also means that `let_async_scope` will be exception safe by default.

§ 6 Design considerations

§ 6.1 Shape of the given sender

§ 6.1.1 Constraints on `set_value()`

It makes sense for `spawn_future()` and `nest()` to accept senders with any type of completion signatures. The caller gets back a sender that can be chained with other senders, and it doesn't make sense to restrict the shape of this sender.

The same reasoning doesn't necessarily follow for `spawn()` as it returns `void` and the result of the spawned sender is dropped. There are two main alternatives:

- do not constrain the shape of the input sender (i.e., dropping the results of the computation)
- constrain the shape of the input sender

The current proposal goes with the second alternative. The main reason is to make it more difficult and explicit to silently drop results. The caller can always transform the input sender before passing it to `spawn()` to drop the values manually.

Chosen: `spawn()` accepts only senders that advertise `set_value()` (without any parameters) in the completion signatures.

§ 6.1.2 Handling errors in `spawn()`

The current proposal does not accept senders that can complete with error given to `spawn()`. This will prevent accidental error scenarios that will terminate the application. The user must deal with all possible errors before passing the sender to `spawn()`. i.e., error handling must be explicit.

Another alternative considered was to call `std::terminate()` when the sender completes with error.

Another alternative is to silently drop the errors when receiving them. This is considered bad practice, as it will often lead to first spotting bugs in production.

Chosen: `spawn()` accepts only senders that do not call `set_error()`. Explicit error handling is preferred over stopping the application, and over silently ignoring the error.

§ 6.1.3 Handling stop signals in `spawn()`

Similar to the error case, we have the alternative of allowing or forbidding `set_stopped()` as a completion signal. Because the goal of `counting_scope` is to track the lifetime of the work started through it, it shouldn't matter whether that the work completed with success or by being stopped. As it is assumed that sending the stop signal is the result of an explicit choice, it makes sense to allow senders that can terminate with `set_stopped()`.

The alternative would require transforming the sender before passing it to `spawn`, something like `spawn(std::move(snd) | let_stopped(just), s.get_token())`. This is considered boilerplate and not helpful, as the stopped scenarios should be implicit, and not require handling.

Chosen: `spawn()` accepts senders that complete with `set_stopped()`.

§ 6.1.4 No shape restrictions for the senders passed to `spawn_future()` and `nest()`

Similarly to `spawn()`, we can constrain `spawn_future()` and `nest()` to accept only a limited set of senders. But, because we can attach continuations for these senders, we would be limiting the functionality that can be expressed. For example, the continuation can handle different types of values and errors.

Chosen: `spawn_future()` and `nest()` accept senders with any completion signatures.

§ 6.2 P2300's `start_detached()`

The `spawn()` algorithm in this paper can be used as a replacement for `start_detached` proposed in [P2300R7]. Essentially it does the same thing, but it also provides the given scope the opportunity to apply its bookkeeping policy to the given sender, which, in the case of `counting_scope`, ensures the program can wait for spawned work to complete before destroying any resources references by that work.

§ 6.3 P2300's `ensure_started()`

The `spawn_future()` algorithm in this paper can be used as a replacement for `ensure_started` proposed in [P2300R7]. Essentially it does the same thing, but it also provides the given scope the opportunity to apply its bookkeeping policy to the given sender, which, in the case of `counting_scope`, ensures the program can wait for spawned work to complete before destroying any resources references by that work.

§ 6.4 Supporting the pipe operator

This paper doesn't support the pipe operator to be used in conjunction with `spawn()` and `spawn_future()`. One might think that it is useful to write code like the following:

```
std::move(snd1) | spawn(s); // returns void
sender auto snd3 = std::move(snd2) | spawn_future(s) | then(...);
```

In [P2300R7] sender consumers do not have support for the pipe operator. As `spawn()` works similarly to `start_detached()` from [P2300R7], which is a sender consumer, if we follow the same rationale, it makes sense not to support the pipe operator for `spawn()`.

On the other hand, `spawn_future()` is not a sender consumer, thus we might have considered adding pipe operator to it.

On the third hand, Unifex supports the pipe operator for both of its equivalent algorithms (`unifex::spawn_detached()` and `unifex::spawn_future()`) and Unifex users have not been confused by this choice.

To keep consistency with `spawn()` this paper doesn't support pipe operator for `spawn_future()`.

§ 7 Naming

As is often true, naming is a difficult task. We feel more confident about having arrived at a reasonably good naming *scheme* than good *names*:

- There is some consensus that the default standard “scope” should be the one this paper calls `counting_scope` because it provides all of the obviously-useful features of a scope, while `simple_counting_scope` is the more spare type that only provides scoping facilities. Therefore, `counting_scope` should get the “nice” name, while `simple_counting_scope` should get a more cumbersome name that conveys fewer features in exchange for a smaller object size and fewer atomic operations.
- Most people seem to hate the name `counting_scope` because the “counting” is an implementation detail, there are arguments about whether it's really “scoping” anything, and the name doesn't really tell you what the type is *for*. The leading suggestion for a better

name is to pick one that conveys that the type “groups together” or “keeps track of” “tasks”, “senders”, or “operations”. Examples of this scheme include `task_pool`, `sender_group`, and `task_arena`. We like the suggested pattern but seek LEWG’s feedback on:

- Should we choose `task` or `sender` to describe the thing being “grouped”? `task` feels friendlier, but might risk conveying that not all sender types are supported.
- What word should we use to describe the “grouping”?
 - `pool` often means a pre-allocated group of resources that can be borrowed from and returned to, which isn’t appropriate.
 - `group` is either the most generic word for a group of things, or an unrelated mathematical object.
 - `arena` is used outside computing to mean a place where competitions happen, and within computing to refer to a memory allocation strategy.
 - Something else?
- The name-part token was selected by analogy to `stop_token`, but it feels like a loose analogy. Perhaps `handle` or `ref` (short for reference) would be better. `ref` is nice for being short and accurate.
- The likely use of the `async_scope_token` concept will be to constrain algorithms that accept a sender and a token with code like the following:

```
template <sender Sender, async_scope_token Token>
void foo(Sender, Token);
```

We propose the token concept should be named `async_<new name of counting_scope><new word for token>`. Assuming we choose `task_pool` and `ref`, that would produce `async_task_pool_ref`, which would look like this:

```
template <sender Sender, async_task_pool_ref Ref>
void foo(Sender, Ref);
```

- The simple prefix does not convey much about how `simple_counting_scope` is “simple”. Suggestions for alternatives include:
 - `fast` by analogy to the fast-prefixed standard integer types, which are so-named because they’re expected to be efficient.
 - `non_cancellable` to speak to what’s “missing” relative to `counting_scope`, however, `simple_counting_scope` does not change the cancellability of senders nested within it and we worry that this suggestion might convey that senders nested within a `non_cancellable` scope might somehow *lose* cancellability.

§ 7.1 `async_scope_token`

This is a concept that is satisfied by types that support associating senders with scopes. It is primarily useful for constraining the arguments to scope-related algorithms like the proposed `nest()`, `spawn()`, and `spawn_future()` to give useful error messages for invalid invocations.

alternatives: `task_pool_ref`, `task_pool_token`, `task_group_ref`, `sender_group_ref`, `task_group_token`, `sender_group_token`, don’t name it and leave it as *exposition-only*

§ 7.2 `nest()`

This provides a way to build a sender that is associated with a “scope”, which is a type that implements and enforces some bookkeeping policy regarding the senders nested within it. `nest()` does not allocate state, call `connect`, or call `start`.

It would be good for the name to indicate that it is a simple operation (insert, add, embed, extend might communicate allocation, which `nest()` does not do).

alternatives: `wrap()`, `attach()`, `track()`, `add()`, `associate()`

§ 7.3 `spawn()`

This provides a way to start a sender that produces `void` and to associate the resulting async work with an async scope that can implement a bookkeeping policy that may help ensure the async work is complete before destroying any resources it is using. This allocates, connects, and starts the given sender.

It would be good for the name to indicate that it is an expensive operation.

alternatives: `connect_and_start()`, `spawn_detached()`, `fire_and_remember()`

§ 7.4 `spawn_future()`

This provides a way to start work and later ask for the result. This will allocate, connect, and start the given sender, while resolving the race (using synchronization primitives) between the completion of the given sender and the start of the returned sender. Since the type of the receiver supplied to the result sender is not known when the given sender starts, the receiver will be type-erased when it is connected.

It would be good for the name to be ugly, to indicate that it is a more expensive operation than `spawn()`.

alternatives: `spawn_with_result()`

§ 7.5 `simple_counting_scope`

A `simple_counting_scope` represents the root of a set of nested lifetimes.

One mental model for this is a semaphore. It tracks a count of lifetimes and fires an event when the count reaches 0.

Another mental model for this is block syntax. `{}` represents the root of a set of lifetimes of locals and temporaries and nested blocks.

Another mental model for this is a container. This is the least accurate model. This container is a value that does not contain values. This container contains a set of active senders (an active sender is not a value, it is an operation).

alternatives: `simple_async_scope`, `simple_task_pool`, `fast_task_pool`, `non_cancellable_task_pool`, `simple_task_group`, `simple_sender_group`

§ 7.6 `counting_scope`

Has all of the same behavior as `simple_counting_scope`, with the added functionality of cancellation; work nested in this scope can be asked to cancel *en masse* from the scope.

alternatives: `async_scope`, `task_pool`, `task_group`, `sender_group`

§ 7.6.1 `counting_scope::join()`

This method returns a sender that, when started, waits for the scope's count of outstanding senders to drop to zero before completing. It is somewhat analogous to `std::thread::join()` but does not block.

`join()` must be invoked, and the returned sender must be connected, started, and completed, before the scope may be destroyed so it may be useful to convey some of this importance in the name, although `std::thread` has similar requirements for its `join()`.

`join()` is the biggest wart in this design; the need to manually manage the end of a scope's lifetime stands out as less-than-ideal in C++, and there is some real risk that users will write deadlocks with `join()` so perhaps `join()` should have a name that conveys danger.

alternatives: `complete()`, `close()`

§ 8 Specification

§ 8.1 Header `<version>` synopsis 17.3.2 [[version.syn](#)]

To the `<version>` synopsis 17.3.2 [[version.syn](#)], add the following:

```
#define __cpp_lib_coroutine          201902L // also in <coroutine>
#define __cpp_lib_counting_scope    2025XXL // also in <execution>
#define __cpp_lib_debugging         202403L // freestanding, also in <debugging>
```

§ 8.2 Header `<execution>` synopsis 34.4 [[execution.syn](#)]

To the `<execution>` synopsis 34.4 [[execution.syn](#)], add the following after the declaration of `run_loop`:

```
...
namespace std::execution {
    ...
    // [exec.run.loop], run_loop
    class run_loop;

    // [exec.scope.concepts], scope concepts
    template <class Assoc>
        concept async_scope_association = see below;

    template <class Token>
        concept async_scope_token = see below;

    // [exec.scope.expos]
```

```

struct spawn-state-base; // exposition-only

struct spawn-receiver; // exposition-only

template <class Alloc, async_scope_token Token, sender Sender>
    struct spawn-state; // exposition-only

template <class Sigs>
    struct spawn-future-state-base; // exposition-only

template <class Sigs>
    struct spawn-future-receiver; // exposition-only

template <class Alloc, async_scope_token Token, sender Sender>
    struct spawn-future-state; // exposition-only

template <async_scope_token Token, sender Sender>
    using wrapped-sender-from =
        decay_t<decltype(declval<Token&>().wrap(declval<Sender>()))>; // exposition-only

// [exec.scope.algos]
struct nest_t { unspecified };
struct spawn_t { unspecified };
struct spawn_future_t { unspecified };

inline constexpr nest_t nest{};
inline constexpr spawn_t spawn{};
inline constexpr spawn_future_t spawn_future{};

// [exec.simple.counting.scope]
class simple_counting_scope;

// [exec.counting.scope]
class counting_scope;
}

```

§ 8.3 Async scope concepts

Add the following as a new subsection immediately after [exec.utils.tfxcmplsig]:

Scope concepts [exec.scope.concepts]

- 1 The `async_scope_token<Token>` concept defines the requirements on an object of type `Token` that can be used to create associations between senders and an async scope.

```

namespace std::execution {

template <class Token>
concept async_scope_token =
    copyable<Token> &&
    requires(Token token) {
        { token.try_associate() } -> same_as<bool>;
        { token.disassociate() } -> same_as<void>;
    };
}

```

- 2 `async_scope_token<Token>` is modeled only if `Token`'s copy operations, move operations, and `disassociate` method are not potentially throwing.
- 3 For a subexpression `snd`, let `Sndr` be `decltype((snd))` and let `sender<Sndr>` be true; `async_scope_token<Token>` is modeled only if, for an object, `token`, of type `Token`, the expression `token.wrap(snd)` is a valid expression and returns an object that satisfies `sender`.

§ 8.4 `execution::nest`

Add the following as a new subsection immediately after 34.9.12.14 [exec.stopped.err]:

`std::execution::nest` [exec.nest]

- 1 `nest` tries to associate a sender with an async scope such that the scope can track the lifetime of any async operations created with the sender.
- 2 Let `nest-data` be the following exposition-only class template:

```

namespace std::execution {

template <async_scope_token Token, sender Sender>
struct nest_data {
    using wrap_sender = wrapped_sender_from<Token, Sender>;

    optional<wrap_sender> sndr;
    Token token;

    nest_data(Token t, Sender&& s)
        : sndr(t.wrap(std::forward<Sender>(s))),
          token(t) {
        if (!token.try_associate()) {
            sndr.reset();
        }
    }

    nest_data(const nest_data& other) noexcept(is_nothrow_copy_constructible_v<wrap_sender>)
        requires copy_constructible<wrap_sender>
        : token(other.token) {
        if (token.try_associate()) {
            sndr = other.sndr;
        }
    }

    ~nest_data() {
        if (sndr.has_value()) {
            sndr.reset();
            token.disassociate();
        }
    }

    nest_data(nest_data&& other) noexcept(is_nothrow_move_constructible_v<wrap_sender>)
        : sndr(std::move(other.sndr)),
          token(other.token) {
        other.sndr.reset();
    }
};

template <async_scope_token Token, sender Sender>
nest_data(Token, Sender&&) -> nest_data<Token, Sender>;

}

```

- 3 The name *nest* denotes a pipeable sender adaptor object. For subexpressions *sndr* and *token*, if `decltype((sndr))` does not satisfy sender, or `decltype((token))` does not satisfy *async_scope_token*, then `nest(sndr, token)` is ill-formed.

- 4 Otherwise, the expression `nest(sndr, token)` is expression-equivalent to:

```
transform_sender(get_domain_early(sndr), make_sender(nest, nest_data{token, sndr}))
```

except that *sndr* is evaluated only once.

- 5 The exposition-only class template *impls-for* (34.9.1 [exec.snd.general]) is specialized for *nest_t* as follows:

```

namespace std::execution {
    template <>
    struct impls_for<nest_t> : default_impls {
        static constexpr auto get_state = see below;

        static constexpr auto start = see below;
    };
}

```

- 6 The member `impls_for<nest_t>::get_state` is initialized with a callable object equivalent to the following lambda:

```

[]<class Sndr, class Rcvr>(Sndr&& sndr, Rcvr& rcvr) noexcept(see below) {
    auto& [_, data, ...child] = sndr;

    using scope_token = decltype(data.token);
    using op_t = decltype(connect(std::forward_like<Sndr>(data.sndr.value()), rcvr));

    static_assert(sizeof...(child) == 0);

    struct op_state {
        bool associated(false);
        scope_token token;
        union {

```

```

    Rcvr* rcvr;
    op_t op;
};

op_state(scope_token token, Rcvr& rcvr) noexcept
: token(std::move(token)),
  rcvr(&rcvr) {}

op_state(scope_token token, wrap_sender&& sndr, Rcvr& rcvr)
: associated(true),
  token(std::move(token)),
  op(connect(std::move(sndr), std::move(rcvr))) {}

op_state(scope_token token, const wrap_sender& sndr, Rcvr& rcvr)
: associated(token.try_associate()),
  token(std::move(token)),
  rcvr(&rcvr) {
    if (associated) {
        new (&op) op_t(connect(sndr, std::move(rcvr)));
    }
}

op_state(op_state&&) = delete;

~op_state() {
    if (associated) {
        op.~op_t();
    }
}

void start() {
    if (associated) {
        op.start();
    }
    else {
        set_stopped(std::move(rcvr));
    }
}

};

if (associated) {
    return op_state{std::forward_like<Sndr>(data.token), std::forward_like<Sndr>(data.sndr.value()), rcvr};
}
else {
    return op_state{data.token, rcvr};
}
}

```

7 The member `impls-for<nest_t>::start` is initialized with a callable object equivalent to the following lambda:

```

[] (auto& state, auto&) noexcept -> void {
    state.start();
}

```

8 The evaluation of `nest(sndr, token)` may cause side effects observable via `token`'s associated async scope object.

§ 8.5 `execution::spawn`

Add the following as a new subsection immediately after `[exec.nest]`:

`std::execution::spawn` `[exec.scope.spawn]`

- 1 `spawn` attempts to associate the given input sender with the given token's async scope and, on success, eagerly starts the input sender.
- 2 The name `spawn` denotes a customization point object. For subexpressions `sndr`, `token`, and `env`, let `Sndr` be `decltype((sndr))`, let `Token` be `decltype((token))`, and let `Env` be `decltype((env))`. If `sender<Sndr>` or `async_scope_token<Token>` is false, the expression `spawn(sndr, token, env)` is ill-formed.
- 3 For the expression `spawn(sndr, token, env)` let `new-sender` be the expression `token.wrap(sndr)` and let `alloc` and `senv` be defined as follows:
 - if the expression `get_allocator(env)` is well defined, then `alloc` is the result of `get_allocator(env)` and `senv` is the expression `env`,
 - otherwise if the expression `get_allocator(get_env(new-sender))` is well-defined, then `alloc` is the result of `get_allocator(get_env(new-sender))` and `senv` is the expression `JOIN-ENV(env, MAKE-ENV(get_allocator, alloc))`

— otherwise `alloc` is `std::allocator<void>{}` and `send` is the expression `env`

- 4 Let *spawn-state-base* be an exposition only class defined below:

```
namespace std::execution {  
  
struct spawn-state-base { // exposition-only  
    virtual void complete() = 0; // exposition-only  
};  
  
}
```

- 5 Let *spawn-receiver* be an exposition only class defined below:

```
namespace std::execution {  
  
struct spawn-receiver { // exposition-only  
    spawn-state-base* state; // exposition-only  
    void set_value() && noexcept { state->complete(); }  
    void set_stopped() && noexcept { state->complete(); }  
};  
  
}
```

- 6 Let *spawn-state* be an exposition only class template defined below:

```
namespace std::execution {  
  
template <class Alloc, async_scope_token Token, sender Sender>  
struct spawn-state : spawn-state-base {  
    using op-t = decltype(connect(declval<Sender>(), spawn-receiver{nullptr}));  
  
    spawn-state(Alloc alloc, Sender&& sndr, Token token); // see below  
    void run(); // see below  
    void complete() override; // see below  
  
private:  
    using alloc-t = typename allocator_traits<Alloc>::template rebind_alloc<spawn-state>;  
  
    alloc-t alloc;  
    op-t op;  
    Token token;  
};  
  
}
```

```
spawn-state(Alloc alloc, Sender&& sndr, Token token);
```

- 7 *Effects*: Equivalent to:

```
this->alloc = alloc;  
this->op = connect(std::move(sndr), spawn-receiver{this});  
this->token = token;
```

```
void run();
```

- 9 *Effects*: Equivalent to:

```
if (token.try_associate()) {  
    op.start()  
} else {  
    complete();  
}
```

```
void complete() override;
```

- 10 *Effects*: Equivalent to:

```
auto token = this->token;  
{  
    auto alloc = std::move(this->alloc);  
  
    allocator_traits<alloc-t>::destroy(alloc, this);  
    allocator_traits<alloc-t>::deallocate(alloc, this, 1);  
}  
token.disassociate();
```

- 11 Then the expression `spawn(sndr, token)` is expression-equivalent to `spawn(sndr, token, empty_env{})` and the expression `spawn(sndr, token, env)` is expression-equivalent to the following:

```
auto makeSender = [&] {
    return write_env(token.wrap(std::forward<Sender>(sndr)), sendv);
};

using sender_t = decltype(makeSender());

using state_t = spawn_state<decltype(alloc), Token, sender_t>;
using alloc_t = typename allocator_traits<decltype(alloc)>::template rebind_alloc<state_t>;
using traits_t = allocator_traits<alloc_t>;

alloc_t stateAlloc{alloc};
auto* op = traits_t::allocate(stateAlloc, 1);

try {
    traits_t::construct(stateAlloc, op, alloc, makeSender(), token);
}
catch(...) {
    traits_t::deallocate(stateAlloc, op, 1);
    throw;
}

try {
    op->run();
}
catch(...) {
    traits_t::destroy(stateAlloc, op);
    traits_t::deallocate(stateAlloc, op, 1);
    throw;
}
```

§ 8.6 `execution::spawn_future`

Add the following as a new subsection immediately after `[exec.scope.spawn]`:

`std::execution::spawn_future` `[exec.scope.spawn.future]`

- 1 `spawn_future` attempts to associate the given input sender with the given token's async scope and, on success, eagerly starts the input sender; the return value is a sender that, when connected and started, completes with either the result of the eagerly-started input sender or with `set_stopped` if the input sender was not started.
- 2 The name `spawn_future` denotes a customization point object. For subexpressions `sndr`, `token`, and `env`, let `Sndr` be `decltype((sndr))`, let `Token` be `decltype((token))`, and let `Env` be `decltype((env))`. If `sender<Sndr>` or `async_scope_token<Token>` is false, the expression `spawn_future(sndr, token, env)` is ill-formed.
- 3 For the expression `spawn_future(sndr, token, env)` let `stok` be a stop token that will receive stop requests as follows:
 - if `get_stop_token(env)` is well defined then `stok` receives stop requests sent from the returned future and any stop requests sent to the stop token returned from `get_stop_token(env)`;
 - otherwise `stok` only receives stop requests sent from the returned future.
- 4 For the expression `spawn_future(sndr, token, env)` let `new_sender` be the expression `token.wrap(sndr)` and let `alloc` and `sendv` be defined as follows:
 - if the expression `get_allocator(env)` is well defined, then `alloc` is the result of `get_allocator(env)` and `sendv` is the expression `JOIN-ENV(env, MAKE-ENV(get_stop_token, stok))`;
 - otherwise if the expression `get_allocator(get_env(new_sender))` is well-defined, then `alloc` is the result of `get_allocator(get_env(new_sender))` and `sendv` is the expression `JOIN-ENV(env, MAKE-ENV(get_allocator, alloc), MAKE-ENV(get_stop_token, stok))`;
 - otherwise `alloc` is `std::allocator<void>` and `sendv` is the expression `JOIN-ENV(env, MAKE-ENV(get_stop_token, stok))`.
- 5 Let `spawn_future_state_base` be an exposition-only class template defined below:

```
namespace std::execution {

template <class Sigs>
struct spawn_future_state_base { // exposition-only
    variant</* see below */> result; // exposition-only
    virtual void complete() = 0; // exposition-only
};
```

```
}
```

- 6 The class template *spawn-future-state-base* can be instantiated with a type parameter, *Sigs*, that is an instantiation of *completion_signatures*. For an instantiation of *spawn-future-state-base*, the result member has the type *variant<T...>* where the parameter pack contains the following:

- *monostate* as the first element;
- for each completion signature in *Sigs* with a completion tag *cpo_t* and parameter types *P...* an element of type *tuple<cpo_t, remove_cvref_t<P>...>*; and
- *tuple<set_error_t, exception_ptr>* if any of the preceding instantiations of *tuple* have possibly-throwing constructors.

- 7 Let *spawn-future-receiver* be an exposition-only class template defined below:

```
namespace std::execution {

template <class Sigs>
struct spawn-future-receiver { // exposition-only
    spawn-future-state-base<Sigs>* state; // exposition-only

    template <class... T>
    void set_value(T&&... t) && noexcept {
        try {
            state->result.emplace<tuple<set_value_t, remove_cvref_t<T>...>>(set_value, std::forward<T>(t)...);
        }
        catch (...) {
            state->result.emplace<tuple<set_error_t, exception_ptr>>(set_error, std::current_exception());
        }
        state->complete();
    }

    template <class E>
    void set_error(E&& e) && noexcept {
        try {
            state->result.emplace<tuple<set_error_t, remove_cvref_t<E>>>(set_error, std::forward<E>(e));
        }
        catch (...) {
            state->result.emplace<tuple<set_error_t, exception_ptr>>(set_error, current_exception());
        }
        state->complete();
    }

    void set_stopped() && noexcept {
        state->result.emplace<tuple<set_stopped_t>>(set_stopped);
        state->complete();
    }
};

}
```

- 8 Let *spawn-future-state* be an exposition-only class template defined below:

```
namespace std::execution {

template <class Alloc, async_scope_token Token, sender Sender>
struct spawn-future-state : spawn-future-state-base<completion_signatures_of_t<Sender, empty_env>> {
    using sigs-t = completion_signatures_of_t<Sender, empty_env>;
    using receiver-t = spawn-future-receiver<sigs-t>;
    using op-t = decltype(connect(declval<Sender>(), receiver-t{nullptr}));

    spawn-future-state(Alloc alloc, Sender&& sndr, Token token); // see below
    void run(); // see below
    void complete() override; // see below
    void consume(receiver auto& rcvr) noexcept; // see below
    void abandon() noexcept; // see below

private:
    using alloc-t = typename allocator_traits<Alloc>::template rebind_alloc<spawn-future-state>;

    alloc-t alloc;
    op-t op;
    Token token;

    void destroy() noexcept; // see below
};

}
```

```
}
```

```
spawn-future-state(Alloc alloc, Sender&& sndr, Token token);
```

9 *Effects*: Equivalent to:

```
this->alloc = alloc;
this->op = connect(std::move(sndr), spawn-future-receiver<sigs-t>{this});
this->token = token;
```

```
void run();
```

10 *Effects*: Equivalent to:

```
if (token.try_associate()) {
    op.start();
} else {
    this->result.emplace<tuple<set_stopped_t>>(set_stopped);
    complete();
}
```

```
void complete();
```

11 *Effects*:

- If the invocation of *complete* happens-before an invocation of *consume* or *abandon* then no effect;
- otherwise, if an invocation of *consume* happened-before this invocation of *complete* then
 - there is a receiver, *rcvr*, registered and that receiver is completed as if by:

```
std::move(this->result).visit([](auto cpo, auto&&... vals) {
    cpo(std::move(rcvr), std::move(vals)...);
});
```

- then *this->destroy()* is invoked.

- otherwise, an invocation of *abandon* happened-before this invocation of *complete* and *this->destroy()* is invoked.

```
void consume(receiver auto& rcvr) noexcept;
```

12 *Effects*:

- If the invocation of *consume* happens-before an invocation of *complete* then *rcvr* is registered to be completed when *complete* is invoked;
- otherwise, *rcvr* is completed as if by:

```
std::move(this->result).visit([](auto cpo, auto&&... vals) {
    cpo(std::move(rcvr), std::move(vals)...);
});
```

and then *this->destroy()* is invoked.

```
void abandon() noexcept;
```

13 *Effects*:

- If the invocation of *abandon* happens-before an invocation of *complete* then a stop request is sent to the spawned operation;
- otherwise *this->destroy()* is invoked.

```
void destroy() noexcept;
```

14 *Effects*: Equivalent to:

```
auto token = this->token;
{
    auto alloc = std::move(this->alloc);

    allocator_traits<alloc-t>::destroy(alloc, this);
    allocator_traits<alloc-t>::deallocate(alloc, this, 1);
}
token.disassociate();
```

15 The exposition-only class template *impls-for* ([exec.snd.general]) is specialized for *spawn_future_t* as follows:

```

namespace std::execution {

template <>
struct impls-for<spawn_future_t> : default-impls {
    static constexpr auto start = see below;
};

}

```

- 16 The member `impls-for<spawn_future_t>::start` is initialized with a callable object equivalent to the following lambda:

```

[](auto& state, auto& rcvr) noexcept -> void {
    state.release()->consume(rcvr);
}

```

- 17 Then the expression `spawn_future(sndr, token)` is expression-equivalent to `spawn_future(sndr, token, empty_env{})` and the expression `spawn_future(sndr, token, env)` is expression-equivalent to the following:

```

auto makeSender = [&] {
    return write-env(token.wrap(std::forward<Sender>(sndr)), sendv);
};

using sender-t = decltype(makeSender());

using state-t = spawn-future-state<decltype(alloc), Token, sender-t>;
using alloc-t = typename allocator_traits<decltype(alloc)>::template rebind_alloc<state-t>;
using traits-t = allocator_traits<alloc-t>;

alloc-t stateAlloc{alloc};
auto* op = traits-t::allocate(stateAlloc, 1);

try {
    traits-t::construct(stateAlloc, op, alloc, makeSender(), token);
}
catch(...) {
    traits-t::deallocate(stateAlloc, op, 1);
    throw;
}

try {
    op->run();
}
catch(...) {
    traits-t::destroy(stateAlloc, op);
    traits-t::deallocate(stateAlloc, op, 1);
    throw;
}

struct deleter {
    void operator()(state-t p) noexcept {
        if (p) {
            p->abandon();
        }
    }
};

return make-sender(spawn_future, unique_ptr<state-t, deleter>{op});

```

§ 8.7 `execution::simple_counting_scope` and `execution::counting_scope`

Add the following new section immediately after **TODO**:

Scopes [exec.scopes]

Simple Counting Scope [exec.simple.counting.scope]

General [exec.simple.counting.general]

```

class simple_counting_scope {
public:
    // [exec.simple.counting.token], token
    struct token {
        template <sender Sender>
        Sender&& wrap(Sender&& snd) const noexcept;
        bool try_associate() const;
        void disassociate() const;
    };
};

```

```

private:
    simple_counting_scope* scope; // exposition-only
};

struct join_t; // exposition-only

enum state_type { // exposition-only
    unused, // exposition-only
    open, // exposition-only
    close, // exposition-only
    open-and-joining, // exposition-only
    closed-and-joining, // exposition-only
    unused-and-closed, // exposition-only
    joined, // exposition-only
};

// [exec.simple.counting.ctor], constructor and destructor
simple_counting_scope() noexcept;
simple_counting_scope(simple_counting_scope&&) = delete;
~simple_counting_scope();

// [exec.simple.counting.mem], members
token get_token() noexcept;
void close() noexcept;
auto join() noexcept;

private:
    size_t count; // exposition-only
    state_type state; // exposition-only
};

```

- ¹ A `simple_counting_scope` maintains a count of outstanding operations. Let `s` be an object of type `simple_counting_scope`, `t` be an object of type `simple_counting_scope::token` obtained from `s.get_token()`, let `j` be a sender obtained from `s.join()`, and let `o` be an operation state obtained from connecting `j` to a receiver. During its life-time `s` goes through different states which govern what operations are allowed and the result of these operations:

- (1.1) — *unused*: a newly constructed object starts in the *unused* state.
- (1.2) — *open*: when `t.try_associate()` is called while `s` is in *unused* state, `s` moves to the *open* state.
- (1.3) — *open-and-joining*: when the operation state `o` is started while the `s` is in *unused* or *open* state, `s` moves to the *open-and-joining* state.
- (1.4) — *closed*: when `s.close()` is called while `s` is in *open* state, `s` moves to the *closed* state.
- (1.5) — *unused-and-closed*: when `s.close()` is called while `s` is in *unused* state, `s` moves to the *unused-and-closed* state.
- (1.6) — *closed-and-joining*: when `s.close()` is called while `s` is in *open-and-joining* state or the operation state `o` is started while `s` is in *closed* or *unused-and-closed* state, `s` moves to the *closed-and-joining* state.
- (1.7) — *joined*: when the count of associated objects drops to zero while `s` is in *open-and-joining* or *closed-and-joining* state, `s` moves to the *joined* state.

- ² Calls to member functions `get_token`, `close`, and `join` do not introduce data races.

Constructor and Destructor [exec.simple.counting.ctor]

```
simple_counting_scope() noexcept;
```

- ¹ *Postconditions*: `count` is 0 and `state` is *unused*

```
~simple_counting_scope();
```

- ² *Effects*: If `state` is not one of *joined*, *unused*, or *unused-and-closed*, invokes `terminate` (14.6.2 [except.terminate]). Otherwise, has no effects.

Members [exec.simple.counting.mem]

```
token get_token() noexcept;
```

- ¹ *Returns*: An object `t` of type `simple_counting_scope::token` such that `t.scope == this` is true.

```
void close() noexcept;
```

- ² *Effects*: If `state` is

- (2.1) — *unused* changes `state` to *unused-and-closed*;

(2.2) — *open* changes *state* to *closed*;

(2.3) — *open-and-joining* changes *state* to *closed-and-joining*;

3 Any call to `t.try_associate()` for a token object `t` referring to a `simple_counting_scope` object `s` which happens after a call to `s.close()` returns false.

`sender auto join() noexcept;`

4 *Returns:* `make_sender(join-t, this)`

5 The exposition-only class template *impls-for* (34.9.1 [exec.snd.general]) is specialized for *join-t* as follows:

```
template <>
struct impls-for<join-t>: default-impls {
    template <class Receiver>
    struct state { // exposition-only
        simple_counting_scope* scope; // exposition-only
        remove_cvref_t<Receiver>& receiver; // exposition-only
        using op_t = decltype(connect(schedule(get_scheduler(get_env(receiver))), receiver)); // exposition-only
        op_t op; // exposition-only

        state(simple_counting_scope* scope, Receiver& receiver) // exposition-only
            : scope(scope),
              receiver(receiver),
              op(connect(schedule(get_scheduler(get_env(receiver))), receiver)) {}

        void complete() { // exposition-only
            op.start();
        }

        void complete-inline() { // exposition-only
            set_value(std::move(receiver));
        }
    };

    static constexpr auto get-state =
        []<class Receiver>(auto&& sender, Receiver& receiver) {
            auto[_ , self] = sender;
            return state<Receiver>(self, receiver);
        };

    static constexpr auto start =
        [](auto& s, auto&) { see-below; };
};
```

7 In the function object used to initialize *impls-for<join-t>::start* let *state* be `s.scope->state`. If *state* is

(7.1) — *unused*, *unused-and-closed*, or *joined*, `s.complete-inline()` is invoked and changes the state of `*s.scope` to *joined*;

(7.2) — *open*, changes the state of `*s.scope` to *open-and-joining*;

(7.3) — *closed*, changes the state of `*s.scope` to *closed-and-joining*;

8 If `s.complete-inline()` was not invoked, registers `s` with `*s.scope` to have `s.complete()` invoked when `s.scope->count` becomes zero.

Token [exec.simple.counting.token]

```
template <sender Sender>
Sender&& wrap(Sender&& snd) const noexcept;
```

1 *Returns:* `std::forward<Sender>(snd)`;

`bool try_associate() const;`

2 *Effects:* A invocation of this member function has the following atomic effect:

(2.1) — If `scope->state` is not one of *unused*, *open*, or *open-and-joining* the operation has no effect;

(2.2) — otherwise increment `scope->count` and if `scope->state == unused` change this value to *open*.

3 *Returns:* true if `scope->count` was incremented, false otherwise.

`void disassociate() const;`

- 4 *Effects:* Decrements *scope->count*. If *scope->count* is zero after decrementing and *scope->state* is *open-and-joining* or *closed-and-joining*, changes the state of **scope* to *joined* and calls *complete()* on all objects registered with **scope*.

- 5 [Note: Calling *complete()* on any registered object may cause **scope* to get destroyed. —End-Note]

Counting Scope [exec.counting.scope]

General [exec.counting.general]

```
class counting_scope {
public:
    // [exec.counting.token], token
    struct token {
        template <sender Sender>
        sender auto wrap(Sender&& snd) const noexcept;
        bool try_associate() const;
        void disassociate() const;

    private:
        counting_scope* scope; // exposition-only
    };

    struct join_t; // exposition-only

    enum state_type { // exposition-only
        unused, // exposition-only
        open, // exposition-only
        close, // exposition-only
        open-and-joining, // exposition-only
        closed-and-joining, // exposition-only
        unused-and-closed, // exposition-only
        joined, // exposition-only
    };

    // [exec.counting.ctor], constructor and destructor
    counting_scope() noexcept;
    counting_scope(counting_scope&&) = delete;
    ~counting_scope();

    // [exec.counting.mem], members
    token get_token() noexcept;
    void close() noexcept;
    auto join() noexcept;
    void request_stop() noexcept;

private:
    size_t count; // exposition-only
    state_type state; // exposition-only
    inplace_stop_source s_source // exposition-only
};
```

- 1 A *counting_scope* maintains a count of outstanding operations. Let *s* be an object of type *counting_scope*, *t* be an object of type *counting_scope::token* obtained from *s.get_token()*, let *j* be a sender obtained from *s.join()*, and let *o* be an operation state obtained from connecting *j* to a receiver. During its life-time *s* goes through different states which govern what operations are allowed and the result of these operations:

- (1.1) — *unused*: a newly constructed object starts in the *unused* state.
- (1.2) — *open*: when *t.try_associate()* is called while *s* is in *unused* state, *s* moves to the *open* state.
- (1.3) — *open-and-joining*: when the operation state *o* is started while the *s* is in *unused* or *open* state, *s* moves to the *open-and-joining* state.
- (1.4) — *closed*: when *s.close()* is called while *s* is in *open* state, *s* moves to the *closed* state.
- (1.5) — *unused-and-closed*: when *s.close()* is called while *s* is in *unused* state, *s* moves to the *unused-and-closed* state.
- (1.6) — *closed-and-joining*: when *s.close()* is called while *s* is in *open-and-joining* state or the operation state *o* is started while *s* is in *closed* or *unused-and-closed* state, *s* moves to the *closed-and-joining* state.
- (1.7) — *joined*: when the count of associated objects drops to zero while *s* is in *open-and-joining* or *closed-and-joining* state, *s* moves to the *joined* state.

- 2 Calls to member functions *get_token*, *close*, *join*, and *request_stop* do not introduce data races.

Constructor and Destructor [exec.counting.ctor]

counting_scope() noexcept;

1 *Postconditions:* *count* is 0 and *state* is *unused*

~counting_scope();

2 *Effects:* If *state* is not one of *joined*, *unused*, or *unused-and-closed*, invokes *terminate* (14.6.2 [except.terminate]). Otherwise, has no effects.

Members [exec.counting.mem]

token get_token() noexcept;

1 *Returns:* An object *t* of type *counting_scope::token* such that *t.scope == this* is true.

void close() noexcept;

2 *Effects:* If *state* is

(2.1) — *unused* changes *state* to *unused-and-closed*;

(2.2) — *open* changes *state* to *closed*;

(2.3) — *open-and-joining* changes *state* to *closed-and-joining*;

3 Any call to *t.try_associate()* for a token object *t* referring to a *counting_scope* object *s* which happens after a call to *s.close()* returns false.

sender auto join() noexcept;

4 *Returns:* *make_sender(join-t, this)*

5 The exposition-only class template *impls-for* (34.9.1 [exec.snd.general]) is specialized for *join-t* as follows:

```
template <>
struct impls-for<join-t>: default-impls {
    template <class Receiver>
    struct state { // exposition-only
        counting_scope* scope; // exposition-only
        remove_cvref_t<Receiver>& receiver; // exposition-only
        using op_t = decltype(connect(schedule(get_scheduler(get_env(receiver))), receiver)); // exposition-only
        op_t op; // exposition-only

        state(counting_scope* scope, Receiver& receiver) // exposition-only
            : scope(scope),
              receiver(receiver),
              op(connect(schedule(get_scheduler(get_env(receiver))), receiver)) {}

        void complete() { // exposition-only
            op.start();
        }

        void complete-inline() { // exposition-only
            set_value(std::move(receiver));
        }
    };

    static constexpr auto get-state =
        []<class Receiver>(auto&& sender, Receiver& receiver) {
            auto[_ , self] = sender;
            return state<Receiver>(self, receiver);
        };

    static constexpr auto start =
        [](auto& s, auto&) { see-below; };
};
```

7 In the function object used to initialize *impls-for<join-t>::start* let *state* be *s.scope->state*. If *state* is

(7.1) — *unused*, *unused-and-closed*, or *joined*, *s.complete-inline()* is invoked and changes the state of **s.scope* to *joined*;

(7.2) — *open*, changes the state of **s.scope* to *open-and-joining*;

(7.3) — *closed*, changes the state of **s.scope* to *closed-and-joining*;

8 If *s.complete-inline()* was not invoked, registers *s* with **s.scope* to have *s.complete()* invoked when *s.scope->count* becomes zero.

```
void request_stop() noexcept
```

9 *Effects:* Calls `s_source.request_stop()`

Token [exec.counting.token]

```
template <sender Sender>
sender auto wrap(Sender&& snd) const noexcept;
```

1 *Returns:* Sender `osnd` from an exposition-only sender algorithm `stop_when(sender auto&& snd, stoppable_token auto stoken)` that maps its input sender, `snd`, such that, when `osnd` is connected to a receiver `r`, the resulting *operation-state* behaves the same as connecting the original sender, `snd`, to `r`, except that the operation will receive a stop request when either the token returned from `get_stop_token(r)` receives a stop request or when `stoken` receives a stop request.

```
bool try_associate() const;
```

2 *Effects:* A invocation of this member function has the following atomic effect:

(2.1) — If `scope->state` is not one of *unused*, *open*, or *open-and-joining* the operation has no effect;

(2.2) — otherwise increment `scope->count` and if `scope->state == unused` change this value to *open*.

3 *Returns:* `true` if `scope->count` was incremented, `false` otherwise.

```
void disassociate() const;
```

4 *Effects:* If `scope->count` is zero after decrementing and `scope->state` is *open-and-joining* or *closed-and-joining*, changes the state of `*scope` to *joined* and calls `complete()` on all objects registered with `*scope`.

5 [Note: Calling `complete()` on any registered object may cause `*scope` to get destroyed. —End-Note]

§ 9 Acknowledgements

Thanks to Daisy Hollman, Nico Josuttis, Zach Laine, Jonathan Müller, and David Sankel for fruitful discussions about regularity.

Thanks to Lewis Baker, Robert Leahy, Dmitry Prokoptsev, Anthony Williams, and everyone else who contributed to discussions leading to this paper.

Thanks to Andrew Royes for unwavering support for the development and deployment of Unifex at Meta and for recognizing the importance of contributing this paper to the C++ Standard.

Thanks to Eric Niebler for the encouragement and support it took to get this paper published.

§ 10 References

[Dahl72] O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare. *Structured Programming*. Academic Press Ltd., 1972.

[folly::coro] folly::coro.

<https://github.com/facebook/folly/tree/main/folly/experimental/coro>

[folly::coro::AsyncScope] folly::coro::AsyncScope.

<https://github.com/facebook/folly/blob/main/folly/experimental/coro/AsyncScope.h>

[io_uring HTTP server] io_uring HTTP server.

https://github.com/facebookexperimental/libunifex/blob/main/examples/linux/http_server_io_uring_test.cpp

[libunifex] libunifex.

<https://github.com/facebookexperimental/libunifex/>

[P2079R2] Lee Howes, Ruslan Arutyunyan, Michael Voss. 2022-01-15. System execution context.

<https://wg21.link/p2079r2>

[P2300R7] Eric Niebler, Michał Dominiak, Georgy Evtushenko, Lewis Baker, Lucian Radu Teodorescu, Lee Howes, Kirk Shoop, Michael Garland, Bryce Adelstein Lelbach. 2023-04-21. ‘std::execution’.

<https://wg21.link/p2300r7>

[P3296R2] Anthony Williams. let_async_scope.

<https://wg21.link/p3296r2>

[rsys] A smaller, faster video calling library for our apps.

<https://engineering.fb.com/2020/12/21/video-engineering/rsys/>

[unifex::v1::async_scope] unifex::v1::async_scope.

https://github.com/facebookexperimental/libunifex/blob/main/include/unifex/v1/async_scope.hpp

[unifex::v2::async_scope] unifex::v2::async_scope.

https://github.com/facebookexperimental/libunifex/blob/main/include/unifex/v2/async_scope.hpp